

Hydra HeadV2 Specification: Coordinated Head protocol

DRAFT

Sebastian Nagel `sebastian.nagel@iohk.io`

Sasha Bogicevic `sasha.bogicevic@iohk.io`

Franco Testagrossa `franco.testagrossa@iohk.io`

Daniel Firth `daniel.firth@iohk.io`

Noon van der Silk `noon.vandersilk@iohk.io`

Veronika Romashkina `veronika.romashkina@iohk.io`

August 24, 2026

Contents

1	Introduction	1
2	Protocol Overview	2
2.1	Opening the head	2
2.2	The Coordinated Head protocol	2
2.3	Closing the head	4
2.4	Differences	4
3	Preliminaries	5
3.1	Notation	5
3.2	Public key multi-signature scheme	6
3.3	Extended UTxO	6
3.4	KZG Accumulators	8
4	Protocol Setup	10
5	On-chain Protocol	11
5.1	Init transaction	11
5.2	Deposit Transaction	13
5.3	Recover Transaction	14
5.4	Increment Transaction	15
5.5	Decrement Transaction	17
5.6	Close Transaction	19
5.7	Contest Transaction	21
5.8	Fan-Out Transaction	23
6	Off-Chain Protocol	27
6.1	Assumptions	28
6.2	Notation	29
6.3	Variables	29
6.4	Protocol flow	30
6.5	Rollbacks and protocol changes	33
7	Security (WIP — Iteration 1)	36
7.1	Proofs	37

1 Introduction

This document specifies the 'Coordinated Hydra Head' protocol to be implemented as the first version of Hydra Head on Cardano - **Hydra HeadV1**. The protocol is derived from variants described in the original paper [8], but was further simplified to make a first implementation on Cardano possible.

Note that the format and scope of this document is (currently) also inspired by the paper and hence does not include a definition of the networking protocols or concrete message formats. It

Add:
network
specifi-
cation
(message
formats)

is structured similarly, but focuses on a single variant, and avoids indirections and unnecessary generalizations. The document is kept in sync with the reference implementation available on Github [4].

First, a high-level overview of the protocol and how it differs from legacy variants of the Head protocol is given in Section 2. Relevant definitions and notations are introduced in Section 3, while Section 4 describes protocol setup and assumptions. Then, the actual on-chain transactions of the protocol are defined in Section 5, before the off-chain protocol part specifies behavior of Hydra parties off-chain and ties the knot with on-chain transactions in Section 6. At last, Section 7 gives the security definition, properties and proofs for the Coordinated Head protocol.

2 Protocol Overview

The Hydra Head protocol provides functionality to lock a set of UTxOs on a blockchain, referred to as the *mainchain*, and evolve it inside a so-called off-chain *head*, independently of the mainchain. At any point, the head can be closed with the effect that the locked set of UTxOs on the mainchain is replaced by the latest set of UTxOs inside the head. The protocol guarantees full wealth preservation: no generation of funds can happen off-chain (inside a head) and no responsive honest party involved in a head can ever lose any funds other than by consenting to give them away. In exchange for decreased liveness guarantees (stop any time), it can essentially proceed at network speed under good conditions, thereby reducing latency and increasing throughput. At the same time, the head protocol provides the same capabilities as the mainchain by reusing the same ledger model and transaction formats — making the protocol “isomorphic”.

2.1 Opening the head

To create a head-protocol instance, any party may take the role of an *initiator* and ask other parties, the *head members*, to participate in the head by exchanging public keys and agreeing on other protocol parameters. These public keys are used for both, the authentication of head-related on-chain transactions that are restricted to head members (e.g., a non-member is not allowed to close the head) and for signing off-chain transactions in the head.

The initiator then establishes and directly opens the head by submitting an *init* transaction to the mainchain that contains the Hydra protocol parameters and mints special *participation tokens* (*PT*) identifying the head members. The *init* transaction initializes a state machine (see Fig. 1) that manages the life-cycle of UTxOs in the head. The state machine comprises the three states: *open*, *closed*, and *final*. A *state thread token* (*ST*) minted in *init* marks the head output and ensures contract continuity [6].

The head is opened with an empty UTxO set. Participants can then add funds to the head by creating *deposit* transactions on the mainchain, which are subsequently incorporated into the head via *increment* transactions (see Section 5.2 and 5.4). Once the *open* state is confirmed, the head members start running the off-chain head protocol, which evolves the UTxO set independently of the mainchain.

2.2 The Coordinated Head protocol

The actual Head protocol starts once the head is open on-chain, with an initially empty UTxO set.

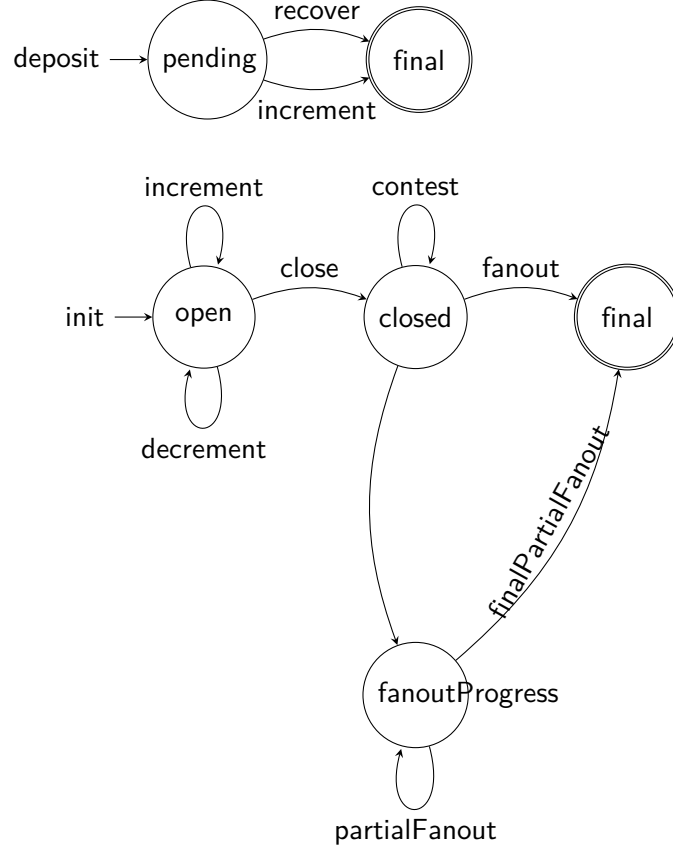


Figure 1: Mainchain state diagram for this version of the Hydra protocol.

The protocol processes off-chain transactions by distributing them between participants, while each party maintains their view of the local UTxO state. That is, the current set of UTxOs evolved from the empty initial state by applying transactions as they are received from the other parties.

To confirm transactions and allow for distribution of the resulting UTxO set without needing the whole transaction history, snapshots are created by the protocol participants. The initial snapshot U_0 corresponds to the initial (empty) UTxO set, while snapshots thereafter $U_1, U_2, \dots$ are created with monotonically increasing snapshot numbers.

For this, the next snapshot leader (round-robin) requests his view of a new confirmed state to be signed by all participants as a new snapshot. The leader does not need to send his local state, but only indicate, by hashes, the set of transactions to be included in order to obtain the to-be-snapshotted UTxO set.

The other participants sign the snapshot as soon as they have (also) seen the transactions that are to be processed on top of its preceding snapshot: a party's local state is always ahead of the latest confirmed snapshot.

Signatures are broadcast and aggregated by each party. When all signature parts of the multi-signature are received and verified, a snapshot is considered confirmed. As a consequence, a participant can safely delete (if wished) all transactions that have been processed into it as the snapshot's multi-signature is now evidence that this state once existed during the head evolution.

2.2.1 Updating an open head

Besides processing “normal” transactions, participants can also request to withdraw some UTxO they can spend from the Head and make it available on main chain via a *decrement* 5.5 transaction — the overall process is also called “decommit”.

To add UTxO to an open head, anyone may create a deposit of one or more UTxO using a *deposit* 5.2 transaction. The head participants will observe this deposit and, once settled, request off-chain agreement to include the deposited UTxO in the form of a snapshot. With that agreement, an *increment* 5.4 transaction can be created and used to update the head state on the mainchain. A deadline is associated with the deposit, which ensures that the UTxO is locked up long enough to be safely consumed into the head without risk of double spending. Should a deposit have not been picked up in time, a *recover* 5.3 transaction allows anyone to unlock the original UTxO.

2.3 Closing the head

The head protocol is designed to allow any head member at any point in time to produce, without interaction, a certificate to close the head. This certificate is created from the latest confirmed, multi-signed snapshot. Using this certificate, the head member may “force close” the head by advancing the mainchain state machine to the *closed* state.

Once in *closed*, the state machine grants parties a contestation period, during which parties may contest the closure by posting the certificate of a newer snapshot on-chain in a contest transaction. Contesting leads back to the state *closed* and each party can contest at most once. After the contestation period has elapsed, the state machine may proceed to the *final* state. The state machine enforces that the outputs of the transaction leading to *final* correspond exactly to the latest UTxO set seen during the contestation period.

2.4 Differences

In the Coordinated Head protocol, off-chain consensus is simplified by not having transactions confirmed concurrently to the snapshots (and to each other) but having the snapshot leader propose, in their snapshot, a set of transactions for explicit confirmation. The parties’ views of confirmed transactions thus progress in sync with each other (once per confirmed snapshot), thus simplifying the close/contest procedure on the mainchain. Also, there is no need for conflict resolution as in Appendix B of [8]. In summary, the differences to the original Head protocol in [8] are:

- No hanging transactions due to ‘coordination’.
- No acknowledgement nor confirmation of transactions.
- No confirmation message for snapshots (two-round local confirmation).
- Allow for deposits and decommits while head is open.
- No initialization phase: head opens directly with empty UTxO, funds added via deposits.

3 Preliminaries

This section introduces notation and other preliminaries used in the remainder of the specification.

3.1 Notation

The specification uses set-notation based approach while also inspired by [1] and [6]. Values a are in a set $a \in \mathcal{A}$, also indicated as being of some type $a : \mathcal{A}$, and multidimensional values are tuples drawn from a $\times$ product of multiple sets, e.g. $(a, b) \in (\mathcal{A} \times \mathcal{B})$. An empty set is indicated by $\emptyset$ and sets may be enumerated using $\{a_1 \dots a_n\}$ notation. The $=$ operator means equality and $\leftarrow$ is explicit assignment of a variable or value to one or more variables. Projection is used to access the elements of a tuple, e.g. $(a, b)^{\downarrow 1} = a$. Functions are morphisms mapping from one set to another $x : \mathcal{A} \rightarrow f(x) : \mathcal{B}$, where function application of a function f to an argument x is written as $f(x)$.

Furthermore, given a set $\mathcal{A}$, let

- $\mathcal{A}^? = \mathcal{A} \cup \diamond$ denotes an option: a value from $\mathcal{A}$ or no value at all indicated by $\perp$,
- $\mathcal{A}^n$ be the set of all n -sized sequences over $\mathcal{A}$,
- $\mathcal{A}^! = \bigcup_{i=1}^{n \in \mathbb{N}} \mathcal{A}^i$ be the set of non-empty sequences over $\mathcal{A}$, and
- $\mathcal{A}^* = \bigcup_{i=0}^{n \in \mathbb{N}} \mathcal{A}^i$ be the set of all sequences over $\mathcal{A}$.

With this, we further define:

- $\mathbb{B} = \{\text{false}, \text{true}\}$ are boolean values
- $\mathbb{N}$ are natural numbers $\{0, 1, 2, \dots\}$
- $\mathbb{Z}$ are integer numbers $\{\dots, -2, -1, 0, 1, 2, \dots\}$
- $\mathbb{H} = \bigcup_{n=0}^{\infty} \{0, 1\}^{8n}$ denotes a arbitrary string of bytes
- $\text{concat} : \mathbb{H}^* \rightarrow \mathbb{H}$ is concatenating bytes, we also use operator $\oplus$ for this
- $\text{hash} : x \rightarrow \mathbb{H}$ denotes a collision-resistant hashing function and $x^\#$ indicates the hash of x
- $\text{bytes} : x \rightarrow \mathbb{H}$ denotes an invertible serialisation function mapping arbitrary data to bytes
- $a||b = \text{concat}(\text{bytes}(a), \text{bytes}(b))$ is an operator which concatenates the $\text{bytes}(b)$ to the $\text{bytes}(a)$
- Lists of values $l \in \mathcal{A}^*$ are written as $l = [x_1, \dots, x_n]$. Empty lists are denoted by $[]$, the i th element x_i is also written $l[i]$ and the length of the list is $|l| = n$. An underscore is also used to indicate a list of values $\underline{x} = l$. Projection on lists are mapped to their elements, i.e. $\underline{x}^{\downarrow 1} = [x_1^{\downarrow 1}, \dots, x_n^{\downarrow 1}]$.
- $\text{sortOn} : i \rightarrow \mathcal{A}^* \rightarrow \mathcal{A}^*$ does sort a list of values on the i th projection.
- **Data** is a universal data type of nested sums and products built up recursively from the base types of $\mathbb{Z}$ and $\mathbb{H}$.

3.2 Public key multi-signature scheme

A multisignature scheme is a set of algorithms where

- **MS-Setup** generates public parameters Π , such that
- $(k^{ver}, k^{sig}) \leftarrow \text{MS-KG}(\Pi)$ can be used to generate fresh key pairs,
- $\sigma \leftarrow \text{MS-Sig}(\Pi, k^{sig}, m)$ signs a message m using key k^{sig} ,
- $\tilde{k} \leftarrow \text{MS-AVK}(\Pi, \bar{k})$ aggregates a list of verification keys $\bar{k}$ into a single, aggregate key $\tilde{k}$,
- $\tilde{\sigma} \leftarrow \text{MS-ASig}(\Pi, m, \bar{k}, \bar{\sigma})$ aggregates a list of signatures $\bar{\sigma}$ about message m into a single, aggregate signature $\tilde{\sigma}$.
- $\text{MS-Verify}(\Pi, \tilde{k}, m, \tilde{\sigma}) \in \mathbb{B}$ verifies an aggregate signature $\tilde{\sigma}$ of message m under an aggregate verification key $\tilde{k}$.

The security definition of a multisignature scheme from [9, 11] guarantees that, if $\tilde{k}$ is produced from a tuple of verification keys $\bar{k}$ via **MS-AVK**, then no aggregate signature $\tilde{\sigma}$ can pass verification $\text{MS-Verify}(\tilde{k}, m, \tilde{\sigma})$ unless all honest parties holding keys in $\bar{k}$ signed m .

Note that in the following, we make the parameter Π implicit and leave out the *ver* suffix for verification key such that $k = k^{ver}$ for better readability.

3.3 Extended UTxO

The Hydra Head protocol is specified to work on the so-called Extended UTxO (EUTxO) ledgers like Cardano.

The basis for EUTxO is Bitcoin's UTxO ledger model [5, 12]. Intuitively, it arranges transactions in a directed acyclic graph, such as the one in Figure 2, where boxes represent transactions with (red) inputs to the left and (black) outputs to the right. A dangling (unconnected) output is an *unspent transaction output (UTxO)* — there are two UTxOs in the figure.

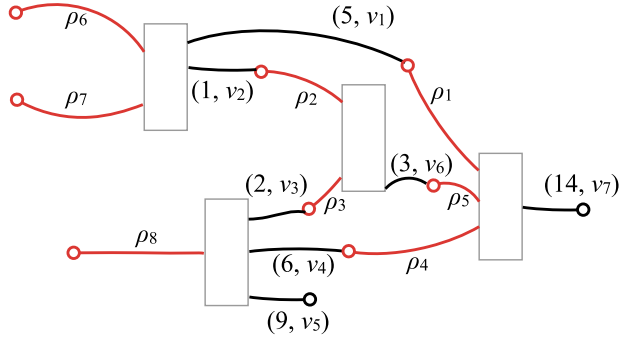


Figure 2: Example of a plain UTxO graph

The following paragraphs will give definitions of the UTxO model and its extension to support scripting (EUTxO) suitable for this Hydra Head protocol specification. For a more detailed introduction to the EUTxO ledger model, see [6], [1] and [7].

3.3.1 Values

Definition 1 (Values). *Values are sets that keep track of how many units of which tokens of which currency are available. Given a finitely supported function $\mapsto$, that maps keys to monoids, a value is the set of such mappings over currencies (minting policy identifiers), over a mapping of token names t to quantities q :*

$$\text{val} \in \text{Val} = (c : \mathbb{H} \mapsto (t : \mathbb{H} \mapsto q : \mathbb{Z}))$$

where addition of values is defined as $+$ and $\emptyset$ is the empty value.

For example, the value $\{c_1 \mapsto \{t_1 \mapsto 1, t_2 \mapsto 1\}\}$ contains tokens t_1 and t_2 of currency c_1 and addition merges currencies and token names naturally:

$$\begin{aligned} & \{c_1 \mapsto \{t_1 \mapsto 1, t_2 \mapsto 1\}\} \\ & + \{c_1 \mapsto \{t_2 \mapsto 1, t_3 \mapsto 1\}, c_2 \mapsto \{t_1 \mapsto 2\}\} \\ & = \{c_1 \mapsto \{t_1 \mapsto 1, t_2 \mapsto 2, t_3 \mapsto 1\}, c_2 \mapsto \{t_1 \mapsto 2\}\}. \end{aligned}$$

While the above definition should be sufficient for the purpose of this specification, a full definition for finitely supported functions and values as used here can be found in [7]. To further improve readability, we define the following shorthands:

- $\{t_1, \dots, t_n\} :: c$ for a set positive single quantity assets $\{c \mapsto \{t_1 \mapsto 1, \dots, t_n \mapsto 1\}\}$,
- $\{t_1, \dots, t_n\}^{-1} :: c$ for a set of negative single quantity assets $\{c \mapsto \{t_1 \mapsto -1, \dots, t_n \mapsto -1\}\}$,
- $\{c \mapsto t \mapsto q\}$ for the value entry $\{c \mapsto \{t \mapsto q\}\}$,
- $\{c \mapsto \cdot \mapsto q\}$ for any asset with currency c and quantity q irrespective of token name.

3.3.2 Scripts

Validator scripts are called *phase-2* scripts in the Cardano Ledger specification (see [3] for a formal treatment of these). Scripts are used for multiple purposes, but most often (and sufficient for this specification) as a *spending* or *minting* policy script.

Definition 2 (Minting Policy Script). *A script $\mu \in \mathcal{M}$ governing whether a value can be minted (or burned), is a pure function with type*

$$\mu \in \mathcal{M} = (\rho : \text{Data}) \rightarrow (\gamma : \Gamma) \rightarrow \mathbb{B},$$

where $\rho \in \text{Data}$ is the redeemer provided as part of the transaction being validated and $\gamma \in \Gamma$ is the validation context.

Definition 3 (Spending Validator Script). *A validator script $\nu \in \mathcal{V}$ governing whether an output can be spent, is a pure function with type*

$$\nu \in \mathcal{V} = (\delta : \text{Data}) \rightarrow (\rho : \text{Data}) \rightarrow (\gamma : \Gamma) \rightarrow \mathbb{B},$$

where $\delta \in \text{Data}$ is the datum available at the output to be spent, $\rho \in \text{Data}$ is the redeemer data provided as part of the transaction being validated, and $\gamma \in \Gamma$ is the validation context.

3.3.3 Transactions

We define EUTxO inputs, outputs and transactions as they are available to scripts and just enough to specify the behavior of the Hydra validator scripts. For example outputs addresses and datums are much more complicated in the full ledger model [1, 2].

actual transactions $\mathcal{T}$ are not defined

Definition 4 (Outputs). An output $o \in \mathcal{O}$ stores some value $\text{val} \in \text{Val}$ at some address, defined by the hash of a validator script $\nu^\# \in \mathbb{H} = \text{hash}(\nu \in \mathcal{V})$, and may store (reference) some data $\delta \in \text{Data}$:

$$o \in \mathcal{O} = (\text{val} : \text{Val} \times \nu^\# : \mathbb{H} \times \delta : \text{Data})$$

Definition 5 (Output references). An output reference $\phi \in \Phi$ points to an output of a transaction, using a transaction id (that is, a hash of the transaction body) and the output index within that transaction.

$$\phi \in \Phi = (\mathbb{H} \times \mathbb{N})$$

Definition 6 (Inputs). A transaction input $i \in \mathcal{I}$ is an output reference $\phi \in \Phi$ with a corresponding redeemer $\rho \in \text{Data}$:

$$i \in \mathcal{I} = (\phi : \Phi \times \rho : \text{Data})$$

Definition 7 (Validation Context). A validation context $\gamma \in \Gamma$ is a view on the transaction to be validated:

$$\gamma \in \Gamma = (\mathcal{I}^* \times \mathcal{O}^* \times \text{Val} \times \mathcal{S}^{\leftrightarrow} \times \mathcal{K})$$

where $\mathcal{I} \in \mathcal{I}^*$ is a **set** of inputs, $\mathcal{O} \in \mathcal{O}^*$ is a **list** of outputs, $\text{mint} \in \text{Val}$ is the minted (or burned) value, $(t_{\min}, t_{\max}) \in \mathcal{S}^{\leftrightarrow}$ are the lower and upper validity bounds where $t_{\min} \leq t_{\max}$, and $\kappa \in \mathcal{K}$ is the set of verification keys which signed the transaction.

Informally, scripts are evaluated by the ledger when it applies a transaction to its current state to yield a new ledger state (besides checking the transaction integrity, signatures and ledger rules). Each validator script referenced by an output is passed its arguments drawn from the output it locks and the transaction context it is executed in. The transaction is valid if and only if all scripts validate, i.e. $\mu(\rho, \gamma) = \text{true}$ and $\nu(\delta, \rho, \gamma) = \text{true}$.

3.3.4 State machines and graphical notation

State machines in the EUTxO ledger model are commonly described using the *constraint emitting machine (CEM)* formalism [6], e.g. the original paper describes the Hydra Head protocol using this notation [8]. Although inspired by CEMs, this specification uses a more direct representation of individual transactions to simplify description of non-state-machine transactions and help translation to concrete implementations on Cardano. The structure of the state machine is enforced on-chain through *scripts* which run as part of the ledger's validation of a transaction (see Section 3.3). For each protocol transaction, the specification defines the structure of the transaction and enumerates the transaction constraints enforced by the scripts ($\text{tx}^\equiv$ in the CEM formalism).

Add an example graph with a legend

3.4 KZG Accumulators

An accumulator scheme over a universe $\mathcal{U}$ is a set of algorithms where

- `accSetup` generates public parameters (the common reference string),

- $\eta \leftarrow \text{accUTxO}(U)$ constructs an accumulator commitment from a UTxO set $U \subseteq \mathcal{U}$,
- $\eta' \leftarrow \text{accCombine}(\eta_1, \eta_2)$ combines two accumulator commitments into one,
- $\pi \leftarrow \text{accWitness}(\eta, S, U \setminus S)$ produces a membership witness π proving that $S \subseteq U$,
- $\text{accVerify}(\eta, S, \pi) \in \mathbb{B}$ verifies a membership witness for subset S ,
- $\eta' \leftarrow \text{accExclude}(\eta, S)$ removes subset S from the accumulator, yielding the updated commitment η' , which itself serves as the exclusion witness, and
- $\text{accVerifyExclude}(\eta, S, \eta') \in \mathbb{B}$ verifies that η' is the correct accumulator after removing S from η .

The security property guarantees that a valid membership witness can only be produced for sets genuinely committed under η , and a valid exclusion witness can only be produced when the subset was genuinely removed.

The implementation uses KZG polynomial commitments [10] over the BLS12-381 elliptic curve.

4 Protocol Setup

In order to create a head-protocol instance, an initiator invites a set of participants (the initiator being one of them) to join by announcing to them the protocol parameters.

- For on-chain transaction authentication (Cardano) purposes, each party p_i generates a corresponding key pair (k_i^{ver}, k_i^{sig}) and sends their verification key k_i^{ver} to all other parties. In the case of Cardano, these are Ed25519 keys.
- For off-chain signing (Hydra) purposes, a very basic multisignature scheme (MS, as defined in Section 3.2) based on EdDSA using Ed25519 keys is used:
 - MS-KG is Ed25519 key generation (requires no parameters)
 - MS-Sign creates an EdDSA signature
 - MS-AVK is concatenation of verification keys into an ordered list
 - MS-ASig is concatenation of signatures into an ordered list
 - MS-Verify verifies the "aggregate" signature by verifying each individual EdDSA signature under the corresponding Ed25519 verification key

To help distinguish on- and off-chain key sets, Cardano verification keys are written k_C , while Hydra verification keys are indicated as k_H for the remainder of this document.

- Each party p_i generates a hydra key pair and sends their hydra verification key to all other parties.
- Each party p_i computes the aggregate key from the received verification keys, stores the aggregate key, their signing key as well as the number of participants n .
- Each party establishes pairwise communication channels to all other parties. That is, every network message received from a specific party is checked for (channel) authentication. It is the implementer's duty to find a suitable authentication process for the communication channels.
- All parties agree on a contestation period $T_{contest}$.

If any of the above fails (or the party does not agree to join the head in the first place), the party aborts the initiation protocol and ignores any further action. Finally, at least one of the participants posts the *init* transaction onchain as described next in Section 5.

5 On-chain Protocol

Update
figures

The following sections describe the *on-chain* protocol controlling the life-cycle of a Hydra head, which can be intuitively described as a state machine (see Figure 1). Each transition in this state machine is represented and caused by a corresponding Hydra protocol transaction on-chain: *init* 5.1, *increment* 5.4, *decrement* 5.5, *close* 5.6, *contest* 5.7, *fanout* 5.8, *partialFanout* 5.8.1, and *finalPartialFanout* 5.8.2.

The protocol uses KZG accumulators (see Section 3.4) to enable partial fanout when UTxO sets exceed transaction size limits. When all UTxOs fit in a single transaction, *fanout* distributes them all at once. When UTxO sets are too large, *partialFanout* distributes subsets across multiple transactions using membership witnesses, transitioning through an intermediate *fanoutProgress* state, until *finalPartialFanout* completes the distribution.

Besides the main state transitions of the head protocol, there is the related “deposit protocol” with two transactions in support of *increment*: *deposit* 5.2 and *recover* 5.3. There is also a *decrement* transaction 5.5 that allows for taking funds from the Head back to L1.

The head protocol defines one minting policy script and one validator script:

- μ_{head} governs minting of state and participation tokens in *init* and burning of these tokens in *fanout*.
- ν_{head} represents the main protocol state machine logic and ensures contract continuity throughout *increment*, *decrement*, *close*, *contest*, *fanout*, *partialFanout* and *finalPartialFanout*.

The deposit protocol defines one validator script:

- ν_{deposit} controls that *deposit* transaction output is claimed correctly into a head via *increment* or recovered after the deadline has passed in a *recover* transaction.

5.1 Init transaction

The *init* transaction creates a head instance and establishes the initial state of the protocol and is shown in Figure 3. The head instance is represented by the unique currency identifier *cid* created by minting tokens using the μ_{head} minting policy script which is parameterized by a single output reference parameter $\phi_{\text{seed}} \in \Phi$:

$$\text{cid} = \text{hash}(\mu_{\text{head}}(\phi_{\text{seed}}))$$

Two kinds of tokens are minted:

- A single *State Thread (ST)* token marking the head output. This output contains the state of the protocol on-chain and the token ensures contract continuity. The token name is the well known string *HydraHeadV2*, i.e. $\text{ST} = \{\text{cid} \mapsto \text{HydraHeadV2} \mapsto 1\}$.
- One *Participation Token (PT)* per participant $i \in \{1 \dots |k_H|\}$ to be used for authenticating further transactions. The token name is the participant’s verification key hash $k_i^\# = \text{hash}(k_i^{\text{ver}})$ of the verification key as received during protocol setup, i.e. $\text{PT}_i = \{\text{cid} \mapsto k_i^\# \mapsto 1\}$.

Update
initTx.svg
to show
direct
Open out-
put with
all tokens.

All minted tokens (ST and all PT_i) are placed directly into the head output, which is created in the *open* state with an empty UTxO set. Consequently, the *init* transaction

- has at least input ϕ_{seed} ,
- mints the state thread token ST, and one PT for each of the $|\underline{k}_{\text{H}}|$ participants with policy cid,
- has one head output o_{head} , which captures the open state of the protocol in the datum

$$\delta_{\text{head}} = (\text{open}, \text{cid}', \phi'_{\text{seed}}, \underline{k}_{\text{H}}, T_{\text{contest}}, v, \eta, \mathbb{A}_{\text{O}})$$

where

- *open* is the state identifier,
- *cid'* is the unique currency id of this instance,
- ϕ'_{seed} is the output reference parameter of μ_{head} ,
- $\underline{k}_{\text{H}}$ are the off-chain multi-signature keys from the setup phase,
- T_{contest} is the contestation period,
- $v = 0$ is the initial snapshot version,
- $\eta = \text{hash}(\emptyset)$ is the hash of the (empty) initial UTxO set.

The $\mu_{\text{head}}(\phi_{\text{seed}})$ minting policy is the only script that verifies init transactions and can be redeemed with either a **Mint** or **Burn** redeemer:

- When evaluated with the **Mint** redeemer,
 1. The seed output is spent in this transaction. This guarantees uniqueness of the policy cid because the EUTxO ledger ensures that ϕ_{seed} cannot be spent twice in the same chain.
 $(\phi_{\text{seed}}, \cdot) \in \mathcal{I}$
 2. All entries of *mint* are of this policy and of single quantity $\forall \{c \mapsto \cdot \mapsto q\} \in \text{mint} : c = \text{cid} \wedge q = 1$
 3. Right number of tokens are minted $|\text{mint}| = |\underline{k}_{\text{H}}| + 1$
 4. State token is sent to the head validator $\text{ST} \in \text{val}_{\text{head}}$
 5. All participation tokens are sent to the head output alongside the state token $\forall i \in [1 \dots |\underline{k}_{\text{H}}|] : \text{PT}_i \in \text{val}_{\text{head}}$
 6. The δ_{head} contains own currency id $\text{cid} = \text{cid}'$ and the right seed reference $\phi_{\text{seed}} = \phi'_{\text{seed}}$
- When evaluated with the **Burn** redeemer,
 1. All tokens for this policy in *mint* need to be of negative quantity $\forall \{\text{cid} \mapsto \cdot \mapsto q\} \in \text{mint} : q < 0$.

Important: The μ_{head} minting policy only ensures uniqueness of cid and that the right amount of tokens have been minted and sent to ν_{head} , while ν_{head} in turn ensures continuity of the contract. However, it is **crucial** that all head members check:

- That the transaction mints exactly the correct tokens: one ST token and one PT for each head member (total $|k_H| + 1$ tokens). This distinguishes *init* from *increment* and *decrement* transactions, which only move tokens without minting.
- That the head output contains an ST token of policy cid which satisfies $\text{cid} = \text{hash}(\mu_{\text{head}}(\phi_{\text{seed}}))$. The ϕ_{seed} from a head datum can be used to determine this.
- That the correct verification key hashes are used in the PTs and the open state is consistent with parameters agreed during setup.

See the initialTx behavior in Figure 13 for details about these checks.

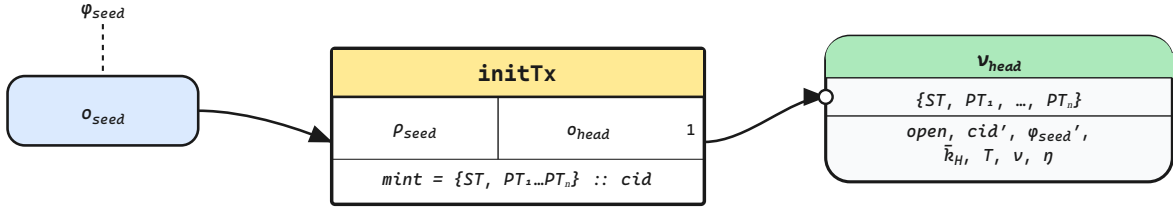


Figure 3: *init* transaction spending a seed UTxO and producing the head output directly in state open.

5.2 Deposit Transaction

The *deposit* transaction locks funds in ν_{deposit} for later collection into the head via an *increment* transaction. Any transaction paying to ν_{deposit} is a *deposit* transaction as there is no on-chain verification in *deposit* transactions. Consequently, protocol actors **must ensure off-chain** that a valid datum is used when paying to the ν_{deposit} validator.

A valid deposit output is governed by ν_{deposit} with value $\text{val}_{\text{deposit}}$ and datum

$$\delta_{\text{deposit}} = (\text{cid}, t_{\text{recover}}, C)$$

where

- cid is the currency id of the target head protocol instance (see 5.1),
- t_{recover} is a deadline after which the deposit can be recovered, and
- $C \in (\mathcal{I} \times \mathbb{H})^*$ is a list of transaction output references with along with serialized outputs that should become available in the head.

Head protocol participants determine **off-chain** whether a deposit output o_{deposit} is eligible for their head by checking

1. cid matches their head identifier,

explain
why this
is enough?

2. t_{recover} is reasonably far in the future, and
3. $\text{val}_{\text{deposit}}$ contains the value of all decoded outputs of C from δ_{deposit} .

explain;
relate to
contesta-
tion pe-
riod?

An example transaction which records all its spent inputs in a deposit output is shown in Figure 4. The $j \in \{1 \dots m\}$ inputs of this example with reference $\phi_{\text{deposited}_j}$ each spend output $o_{\text{deposited}_j}$ with $\text{val}_{\text{deposited}_j}$ would be recorded in the output datum as

$$C = \forall j \in \{1 \dots m\} : [(\phi_{\text{deposited}_j}, \text{bytes}(o_{\text{deposited}_j}))]$$

and the value check would need to satisfy

$$\text{val}_{\text{deposit}} \supseteq \bigcup_{j=1}^m \text{val}_{\text{deposited}_j}$$

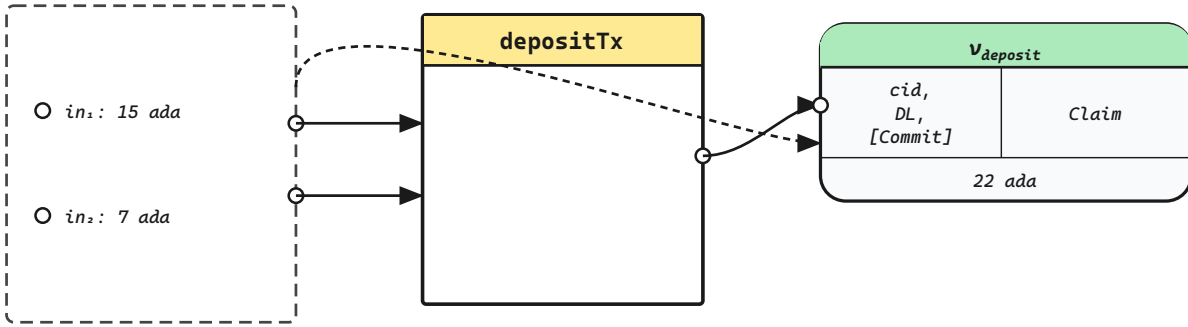


Figure 4: *deposit* transaction spending multiple UTxO into a deposit output.

5.3 Recover Transaction

If a *deposit* transaction output (see 5.2) was not collected into a head by an *increment* transaction 5.4, the *recover* transaction (Figure 5) allows for restoring the UTxO as recorded in the deposit after the deadline has passed. It consists of

- one input spending from ν_{deposit} with datum $\delta_{\text{deposit}} = (\text{cid}, t_{\text{recover}}, C)$, and
- outputs $o_1 \dots o_m$ to recover UTxOs.

The script validator ν_{deposit} is spent with redeemer $\rho_{\text{deposit}} = (\text{Recover}, m)$, where m is the number of outputs to recover, and checks:

1. All UTxOs are recovered exactly as they were deposited. This is done by comparing hashes of serialised representations of the m recovering outputs $o_1 \dots o_m$ with the canonically combined deposited UTxOs in C :

$$\text{hash}\left(\bigoplus_{j=1}^m \text{bytes}(o_j)\right) = \text{hash}(\text{concat}(\text{sortOn}(1, C)^{\downarrow 2}))$$

2. Transaction is posted after the deadline

$$t_{\min} > t_{\text{recover}}$$

3. This deposit is the only one spent by the transaction. The recovered outputs $o_1 \dots o_m$ are the transaction's first m outputs and are therefore shared by every deposit input, so two deposits whose C hashes alike would both be satisfied by a single set of recovered outputs, leaving the second deposit's value to be directed anywhere by the transaction author. Note that ν_{deposit} neither parses the bytes of C nor relates them to the value the deposit holds, so hashing alike is weaker than being equal:

$$|\{(\cdot, o) \in \mathcal{J} \mid o \text{ is governed by } \nu_{\text{deposit}}\}| = 1$$

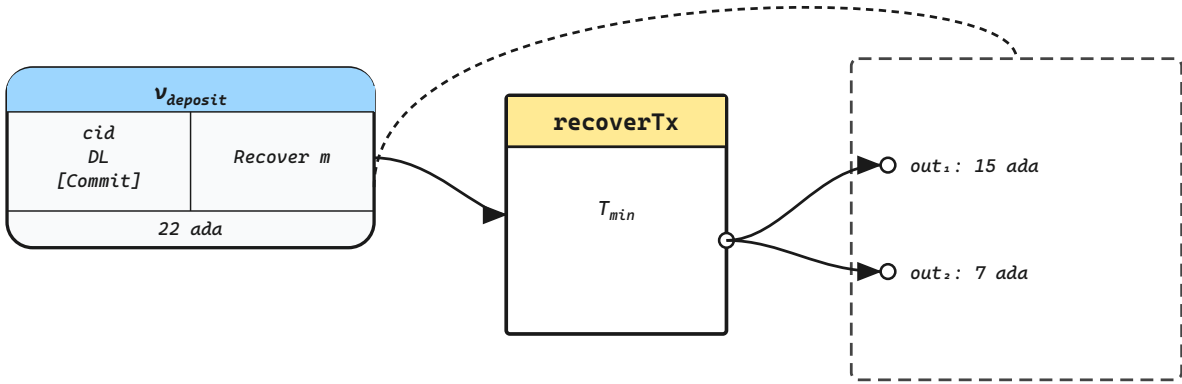


Figure 5: *recover* transaction restoring UTxO of a deposit output.

5.4 Increment Transaction

The *increment* transaction (Figure 6) allows a participant to add a *deposit* output 5.2 to an already open head using a snapshot that approves inclusion. Consequently this transaction consists of:

- one input spending from ν_{head} with value val_{head} holding the ST and datum δ_{head} ,
- one input ϕ_{deposit} spending from ν_{deposit} with value $\text{val}_{\text{deposit}}$ and datum $\delta_{\text{deposit}} = (\text{cid}_{\text{deposit}}, t_{\text{recover}}, C)$,
- one output paying to ν_{head} with value $\text{val}'_{\text{head}}$ and datum δ'_{head} .

The deposit validator ν_{deposit} is spent with $\rho_{\text{deposit}} = \text{Claim}$ and ensures:

1. Claiming head id matches the deposit datum

$$\text{cid} = \text{cid}_{\text{deposit}}$$

2. Transaction is posted before the deadline

$$t_{\max} \leq t_{\text{recover}}$$

3. The head input is spent with an *increment* redeemer that claims this very deposit. Otherwise a legitimate *increment* could carry additional deposit inputs whose value enters the head without any snapshot crediting it:

$$\phi_{\text{increment}} = \phi_{\text{deposit}}$$

The state-machine validator ν_{head} is spent with $\rho_{\text{head}} = (\text{increment}, \xi, s, \phi_{\text{increment}})$, where ξ is a multi-signature of the increment snapshot which authorizes addition of deposited UTxO, s is the snapshot number and ϕ_{deposit} points to the claimed deposit. The validator checks:

1. State is advanced from $\delta_{\text{head}} \sim \text{open}$ to $\delta'_{\text{head}} \sim \text{open}$, parameters $\text{cid}, \tilde{k}_{\text{H}}, n, T_{\text{contest}}$ stay unchanged and the new state is governed again by ν_{head} :

$$(\text{open}, \text{cid}, \tilde{k}_{\text{H}}, n, T_{\text{contest}}, v, \eta, \mathbb{A}_{\text{o}}) \xrightarrow[\xi, s, \phi_{\text{increment}}]{\text{increment}} (\text{open}, \text{cid}, \tilde{k}_{\text{H}}, n, T_{\text{contest}}, v', \eta', \mathbb{A}_{\text{o}})$$

2. New version v' is incremented correctly

$$v' = v + 1$$

3. Claimed deposit is spent

$$\phi_{\text{increment}} = \phi_{\text{deposit}}$$

4. The claimed deposit is the first output of its transaction, as produced by *deposit* 5.2. Since η_{α} below binds a deposit by its transaction id, sibling outputs of that same transaction would otherwise be interchangeable under one signature:

$$\text{txId}(\phi_{\text{deposit}}) = 0$$

5. ξ is a valid multi-signature of the new head state η'

$$\text{MS-Verify}(\underline{k}_{\text{H}}, (\text{cid} || v || s || \eta'^{\#} || \eta_{\omega} || \eta_{\alpha}), \xi) = \text{true}$$

where $\eta'^{\#} = (\eta')^{\#}$ is the hash of the new accumulator commitment η' stored in the output datum, reflecting the UTxO set after adding the deposited UTxOs. The commit digest η_{α} is recomputed from the claimed deposit itself, binding both its recorded outputs C and the identity of the transaction that created it:

$$\eta_{\alpha} = \text{hash}(\text{hash}(\text{concat}(\text{sortOn}(1, C)^{\downarrow 2})) || \text{txId}(\phi_{\text{deposit}}))$$

Hashing C alone would not identify a deposit: a deposit datum is unauthenticated data that anyone can copy into a look-alike deposit holding less value, which hashes identically and would accept the same signature.

6. The value in the head output is increased accordingly

$$\text{val}_{\text{head}} \cup \text{val}_{\text{deposit}} = \text{val}'_{\text{head}}$$

7. Transaction is signed by a participant

$$\exists \{cid \mapsto k_i^\# \mapsto 1\} \in \text{val}'_{\text{head}} \Rightarrow k_i^\# \in \kappa$$

8. The ADA overhead $\mathbb{A}_o$ is preserved across the state transition:

$$\mathbb{A}'_o = \mathbb{A}_o$$

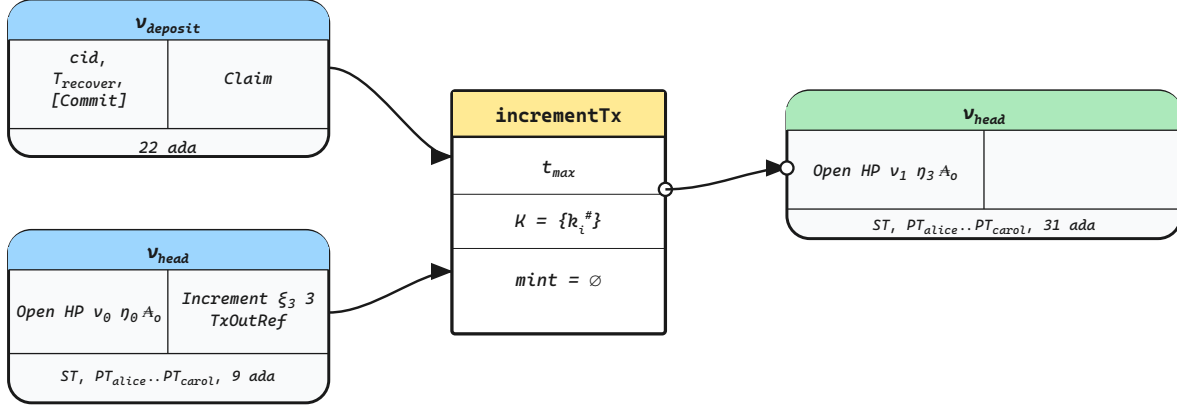


Figure 6: *increment* transaction spending an open head output, producing a new head output which includes the new UTxO.

5.5 Decrement Transaction

The *decrement* transaction (Figure 7) allows a party to remove a UTxO from an already open head and consists of:

- one input spending from ν_{head} holding the ST with δ_{head} ,
- one output paying to ν_{head} with value $\text{val}'_{\text{head}}$ and datum δ'_{head} ,
- one or more decommit outputs $o_2 \dots o_{m+1}$ with values $\text{val}_2 \dots \text{val}_{m+1}$.

The state-machine validator ν_{head} is spent with $\rho_{\text{head}} = (\text{decrement}, \xi, s, m)$, where ξ is a multi-signature of the decrement snapshot which authorizes removal of some UTxO, s is the used snapshot number and m is the number of outputs to distribute. The validator checks:

1. State is advanced from $\delta_{\text{head}} \sim \text{open}$ to $\delta'_{\text{head}} \sim \text{open}$, parameters $\text{cid}, \underline{k}_H, T_{\text{contest}}$ stay unchanged and the new state is governed again by ν_{head}

$$(\text{open}, \text{cid}, \underline{k}_H, T_{\text{contest}}, v, \eta, \mathbb{A}_o) \xrightarrow[\xi, s, m]{\text{decrement}} (\text{open}, \text{cid}, \underline{k}_H, T_{\text{contest}}, v', \eta', \mathbb{A}_o)$$

2. New version v' is incremented correctly

$$v' = v + 1$$

- At least one decommit output is materialized. Note that this is checked on the outputs actually present, not on the redeemer's m : the transaction's decommit outputs are taken as the first m after the head output, which truncates silently when fewer exist.

$$|o_2 \dots o_{m+1}| > 0$$

This is required because *increment* and *decrement* verify the same signed message. With no decommit outputs, η_ω recomputed here is the digest of the empty set — exactly what a snapshot carrying no pending decommit produces — while η_α comes from the redeemer. An *increment* snapshot's multi-signature would otherwise authorize a decrement that advances the version and adopts that snapshot's accumulator, which already counts the deposited UTXOs, while no deposit is spent and no value enters the head. The head would then account for UTXOs it does not hold.

- ξ is a valid multi-signature of the new snapshot state η'

$$\text{MS-Verify}(\underline{k}_H, (\text{cid} || v || s || \eta'^{\#} || \eta_\omega || \eta_\alpha), \xi) = \text{true}$$

where $\eta'^{\#} = (\eta')^{\#}$ is the hash of the new accumulator commitment η' stored in the output datum, reflecting the UTXO set after removing the decommitted UTXOs.

- The value in the head output is decreased accordingly

$$\text{val}'_{\text{head}} \cup \left(\bigcup_{j=2}^{m+1} \text{val}_j \right) = \text{val}_{\text{head}}$$

- Transaction is signed by a participant

$$\exists \{\text{cid} \mapsto k_i^{\#} \mapsto 1\} \in \text{val}'_{\text{head}} \Rightarrow k_i^{\#} \in \kappa$$

- The ADA overhead $\mathbb{A}_o$ is preserved across the state transition:

$$\mathbb{A}'_o = \mathbb{A}_o$$

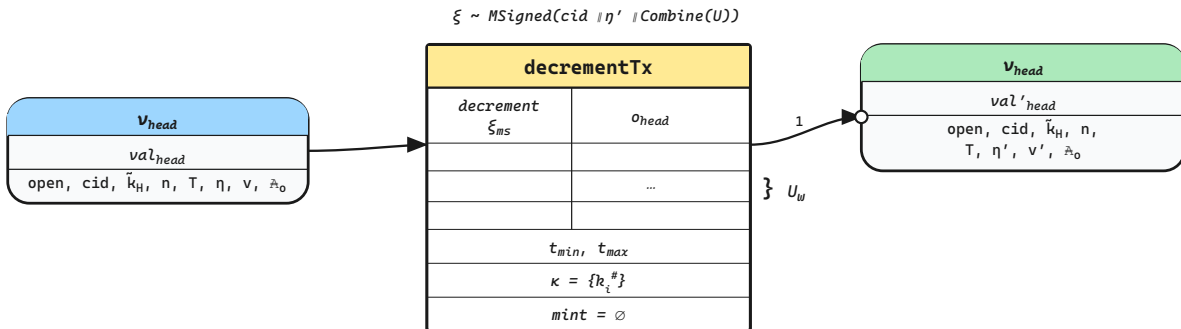


Figure 7: *decrement* transaction spending an open head output, producing a new head output and multiple decommitted outputs.

5.6 Close Transaction

In order to close a head, a head member may post the *close* transaction (see Figure 8). This transaction has

- one input spending from ν_{head} holding the ST with δ_{head} ,
- one output paying to ν_{head} with value $\text{val}'_{\text{head}}$ and datum δ'_{head} .

The state-machine validator ν_{head} is spent with $\rho_{\text{head}} = (\text{close}, \text{CloseType})$, where *CloseType* is a hint against which open state to close and checks:

1. State is advanced from $\delta_{\text{head}} \sim \text{open}$ to $\delta'_{\text{head}} \sim \text{closed}$, parameters $\text{cid}, \underline{k}_{\text{H}}, T_{\text{contest}}$ stay unchanged and the new state is governed again by ν_{head}

$$(\text{open}, \text{cid}, \underline{k}_{\text{H}}, T_{\text{contest}}, v, \eta, \mathbb{A}_{\text{O}}) \xrightarrow[\text{CloseType}]{\text{close}} (\text{closed}, \text{cid}, \underline{k}_{\text{H}}, T_{\text{contest}}, v', s', \eta', \mathcal{C}, t_{\text{final}}, \mathbb{A}_{\text{O}})$$

The closed state carries a single unified accumulator η' that combines the snapshotted UTxO set with any pending increment or decrement UTxOs using *accCombine*.

2. Last known open state version is recorded in closed state

$$v' = v$$

3. Based on the redeemer $\text{CloseType} = \text{Initial} \cup (\text{Any}, \xi, \eta'^{\#}) \cup (\text{Unused}, \xi, \eta'^{\#}) \cup (\text{Used}, \xi, \eta'^{\#})$, where ξ is a multi-signature of the closing snapshot and $\eta'^{\#}$ is the hash of the unified accumulator commitment stored in the output datum, four cases are distinguished. In each case the closed state carries a single unified accumulator η' :

- (a) **Initial:** The initial snapshot is used to close the head and open state was not updated. No signatures are available and it suffices to check

$$v = 0$$

$$s' = 0$$

$$\eta' = \text{accUTxO}(\emptyset)$$

- (b) **Any:** Closing snapshot refers to current state version v with no pending increments or decrements, and $s' > 0$. The unified accumulator is simply the snapshotted state:

$$\eta' = \text{accUTxO}(U')$$

$$\text{MS-Verify}(\underline{k}_{\text{H}}, (\text{cid} || v || s' || \eta'^{\#}), \xi) = \text{true}$$

$$\eta'^{\#} = (\eta')^{\#}$$

- (c) **Unused:** Closing snapshot refers to current state version v and a pending increment or decrement is *not* applied in the snapshot. The unified accumulator is the snapshotted state only:

$$\eta' = \text{accUTxO}(U')$$

$$\text{MS-Verify}(\underline{k}_{\text{H}}, (\text{cid} || v || s' || \eta'^{\#}), \xi) = \text{true}$$

$$\eta'^{\#} = (\eta')^{\#}$$

- (d) **Used:** Closing snapshot refers to the previous state version $v - 1$ and a pending increment or decrement *is* applied in the snapshot. The unified accumulator combines the snapshotted UTxOs with the pending delta:

$$\eta' = \text{accCombine}(\text{accUTxO}(U'), \eta_\Delta)$$

$$\text{MS-Verify}(\underline{k}_H, (\text{cid} || v - 1 || s' || \eta'^{\#}), \xi) = \text{true}$$

$$\eta'^{\#} = (\eta')^{\#}$$

where η_Δ is the accumulator commitment of the pending delta UTxOs.

4. Initializes the set of testers

$$\mathcal{C} = \emptyset$$

This allows the closing party to also contest and is required for use cases where pre-signed, valid in the future, close transactions are used to delegate head closing.

5. Correct contestation deadline is set

$$t_{\text{final}} = t_{\text{max}} + T$$

6. Transaction validity range is bounded by

$$t_{\text{max}} - t_{\text{min}} \leq T$$

to ensure the contestation deadline t_{final} is at most $2 * T$ in the future.

7. Value in the head is preserved

$$\text{val}'_{\text{head}} \supseteq \text{val}_{\text{head}}$$

8. Transaction is signed by a participant

$$\exists \{\text{cid} \mapsto k_i^{\#} \mapsto 1\} \in \text{val}'_{\text{head}} \Rightarrow k_i^{\#} \in \kappa$$

9. No minting or burning

$$\text{mint} = \emptyset$$

10. The ADA overhead $\mathbb{A}_o$ is propagated unchanged from the open datum to the closed datum:

$$\mathbb{A}'_o = \mathbb{A}_o$$

where $\mathbb{A}_o$ is the ADA in the head UTxO not belonging to any L2 UTxO (minimum-UTxO overhead), set at initialisation time and unchanged for the head's lifetime. On fanout, the on-chain value conservation check treats $\mathbb{A}_o$ as released from the head UTxO without requiring it in any distributed output, so it flows to whoever submits the fanout transaction as change (offsetting their transaction fee).

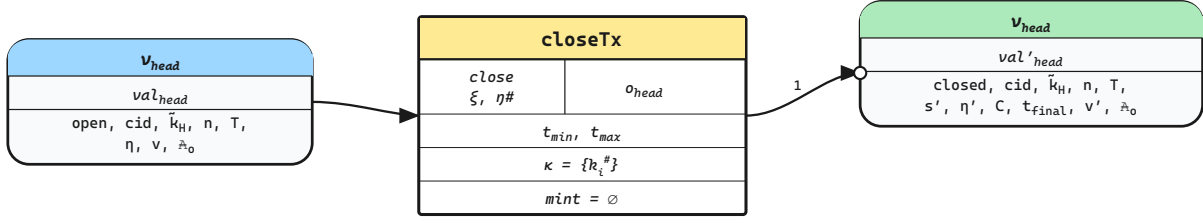


Figure 8: *close* transaction spending the **open** head output and producing a **closed** head output with unified accumulator η' .

5.7 Contest Transaction

The *contest* transaction (see Figure 9) is posted by a party to prove the currently closed state is not the latest one. This transaction has

- one input spending from ν_{head} holding the ST with δ_{head} ,
- one output paying to ν_{head} with value $\text{val}'_{\text{head}}$ and datum δ'_{head} .

The state-machine validator ν_{head} is spent with $\rho_{\text{head}} = (\text{contest}, \text{ContestType})$, where *ContestType* is a hint how to verify the snapshot and checks:

1. State is advanced from $\delta_{\text{head}} \sim \text{open}$ to $\delta'_{\text{head}} \sim \text{closed}$, parameters $\text{cid}, \underline{k}_H, T_{\text{contest}}$ stay unchanged and the new state is governed again by ν_{head}

$$(\text{closed}, \text{cid}, \underline{k}_H, T_{\text{contest}}, v, s, \eta, \mathcal{C}, t_{\text{final}}, \mathbb{A}_0) \xrightarrow[\text{ContestType}]{\text{contest}} (\text{closed}, \text{cid}, \underline{k}_H, T_{\text{contest}}, v', s', \eta', \mathcal{C}', t'_{\text{final}}, \mathbb{A}_0')$$

The closed state carries a single unified accumulator η' computed using *accCombine* based on the contest type.

2. Last known open state version stays recorded in closed state

$$v' = v$$

3. Contested snapshot number s' is higher than the currently stored snapshot number s

$$s' > s$$

4. Based on the redeemer $\text{ContestType} = (\text{Unused}, \xi, \eta'^{\#}) \cup (\text{Used}, \xi, \eta'^{\#})$, where ξ is a multi-signature of the contesting snapshot and $\eta'^{\#} = (\eta')^{\#}$ is the hash of the unified accumulator commitment stored in the output datum, two cases are distinguished:

- (a) **Unused**: Contesting snapshot refers to current state version v (pending delta not applied in snapshot). The unified accumulator reflects only the snapshotted UTXOs:

$$\eta' = \text{accUTxO}(U')$$

$$\text{MS-Verify}(\underline{k}_H, (\text{cid}||v||s'||\eta'^{\#}), \xi) = \text{true}$$

$$\eta'^{\#} = (\eta')^{\#}$$

- (b) **Used:** Contesting snapshot refers to the previous state version $v - 1$ (pending delta applied in snapshot). The unified accumulator combines the snapshotted UTxOs with the pending delta:

$$\eta' = \text{accCombine}(\text{accUTxO}(U'), \eta_{\Delta})$$

$$\text{MS-Verify}(\underline{k}_H, (\text{cid}||v-1||s'||\eta'^{\#}), \xi) = \text{true}$$

$$\eta'^{\#} = (\eta')^{\#}$$

where η_{Δ} is the accumulator commitment of the pending delta UTxOs.

5. The single signer $\{k^{\#}\} = \kappa$ has not already contested and is added to the set of contesters

$$k^{\#} \notin \mathcal{C}$$

$$\mathcal{C}' = \mathcal{C} \cup k^{\#}$$

6. Transaction is posted before deadline

$$t_{\max} \leq t_{\text{final}}$$

7. Contestation deadline is updated correctly to

$$t'_{\text{final}} = \begin{cases} t_{\text{final}} & \text{if } |\mathcal{C}'| = n, \\ t_{\text{final}} + T & \text{otherwise.} \end{cases}$$

8. Value in the head is preserved

$$\text{val}'_{\text{head}} \supseteq \text{val}_{\text{head}}$$

9. Transaction is signed by a participant

$$\exists \{\text{cid} \mapsto k_i^{\#} \mapsto 1\} \in \text{val}'_{\text{head}} \Rightarrow k_i^{\#} \in \kappa$$

10. No minting or burning

$$\text{mint} = \emptyset$$

11. The ADA overhead $\mathbb{A}_o$ is preserved in the output closed datum:

$$\mathbb{A}'_o = \mathbb{A}_o$$

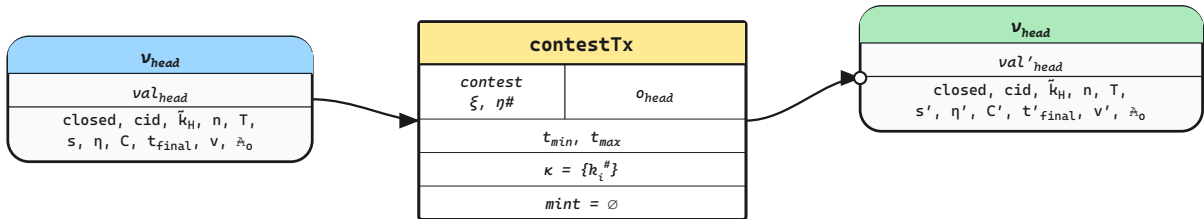


Figure 9: *contest* transaction spending the *closed* head output and producing a different *closed* head output.

5.8 Fan-Out Transaction

Once the contestation phase is over, a head may be finalized by posting a *fanout* transaction (see Figure 10), which distributes all UTxOs from the head according to the unified accumulator in the closed state. A fanout transaction consists of

- one input spending from ν_{head} holding the ST, and
- outputs $o_1 \dots o_m$ to distribute all UTxOs.

The state-machine validator ν_{head} is spent with $\rho_{\text{head}} = (\text{fanout}, m, \pi, \text{crsRef})$, where:

- m is the number of outputs to distribute from the `closed` state,
- π is the KZG membership witness,
- crsRef is the output reference of the reference input holding the Common Reference String (CRS).

The validator checks:

1. State is advanced from $\delta_{\text{head}} \sim \text{closed}$ to terminal state `final`:

$$(\text{closed}, \text{cid}, \underline{k}_{\text{H}}, T_{\text{contest}}, v, s, \eta, \mathcal{C}, t_{\text{final}}) \xrightarrow[m, \pi]{\text{fanout}} \text{final}$$

2. All m outputs are verified as members of the unified accumulator η using the membership witness π :

$$\text{accVerify}(\eta, \{o_1, \dots, o_m\}, \pi) = \text{true}$$

3. Transaction is posted after contestation deadline $t_{\text{min}} > t_{\text{final}}$.
4. All tokens are burnt $|\{\text{cid} \mapsto \cdot \mapsto -1\} \in \text{mint}| = n + 1$.
5. The head input value is fully conserved:

$$\text{val}_{\text{head}}^{\text{in}} = \bigoplus_{i=1}^m \text{val}(o_i) \oplus \text{val}_{\text{burned}} \oplus \mathbb{A}_{\text{o}}$$

where $\mathbb{A}_{\text{o}}$ is the ADA overhead carried from the closed datum, and $\text{val}_{\text{burned}}$ is the value of all burned head tokens.

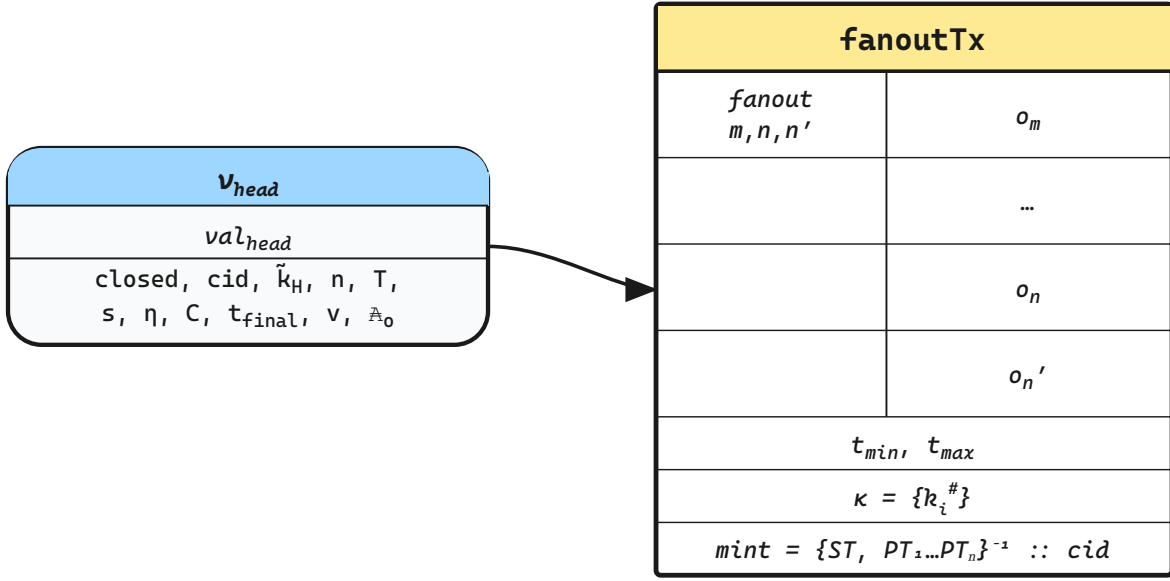


Figure 10: *fanout* transaction spending the closed head output with unified accumulator η and distributing funds with outputs $o_1 \dots o_m$.

5.8.1 Intermediate Partial Fan-Out Transaction

When UTxO sets exceed transaction size limits, the protocol distributes UTxOs across multiple transactions using partial fanout. Each intermediate step distributes a subset of UTxOs and transitions the head into the `fanoutProgress` state, which carries only the fields needed for subsequent steps:

$$(\text{fanoutProgress}, cid, \underline{k}_H, t_{\text{final}}, \eta, \mathbb{A}_o)$$

where t_{final} is the contestation deadline (carried from `closed`), η is the current accumulator commitment, and $\mathbb{A}_o$ is the ADA overhead in the head UTxO not belonging to any L2 UTxO (propagated from the closed datum).

An intermediate partial fanout transaction (see Figure 11) consists of:

- one input spending from ν_{head} in state `closed` or `fanoutProgress`, and
- a continuing head output at index 0 in state `fanoutProgress` with updated accumulator, and
- outputs $o_1 \dots o_m$ at indices $1 \dots m$ distributing a subset of UTxOs.

The state-machine validator ν_{head} is spent with $\rho_{\text{head}} = (\text{partialFanout}, m, \text{crsRef})$, where m is the number of UTxO outputs to distribute in this step and `crsRef` is the output reference of the reference input holding the CRS. The validator checks:

1. $m > 0$ (no zero-output batches).

- State transitions into `fanoutProgress` with updated accumulator. For a closed input:

$$(\text{closed}, \text{cid}, \underline{k}_H, T_{\text{contest}}, v, s, \eta, \mathcal{C}, t_{\text{final}}) \xrightarrow[m]{\text{partialFanout}} (\text{fanoutProgress}, \text{cid}, \underline{k}_H, t_{\text{final}}, \eta', \mathbb{A}_o)$$

For a `fanoutProgress` input:

$$(\text{fanoutProgress}, \text{cid}, \underline{k}_H, t_{\text{final}}, \eta, \mathbb{A}_o) \xrightarrow[m]{\text{partialFanout}} (\text{fanoutProgress}, \text{cid}, \underline{k}_H, t_{\text{final}}, \eta', \mathbb{A}_o)$$

- The new accumulator η' (from the output datum) is not the G1 generator — all elements have *not* yet been removed (use `finalPartialFanout` for the last batch):

$$\eta' \neq G_1$$

- No minting or burning $\text{mint} = \emptyset$.
- Transaction is posted after contestation deadline $t_{\text{min}} > t_{\text{final}}$.
- Parameters `cid`, $\underline{k}_H$, t_{final} , and $\mathbb{A}_o$ are preserved in the output `fanoutProgress` datum.
- Value is conserved: head input value equals head output value plus all distributed outputs:

$$\text{val}_{\text{head}}^{\text{in}} = \text{val}_{\text{head}}^{\text{out}} \oplus \bigoplus_{i=1}^m \text{val}(o_i)$$

- The new accumulator η' from the output datum correctly represents the remaining UTxOs after removing $S = \{o_1, \dots, o_m\}$. The output datum value serves as the exclusion witness:

$$\text{accVerifyExclude}(\eta, S, \eta') = \text{true}$$

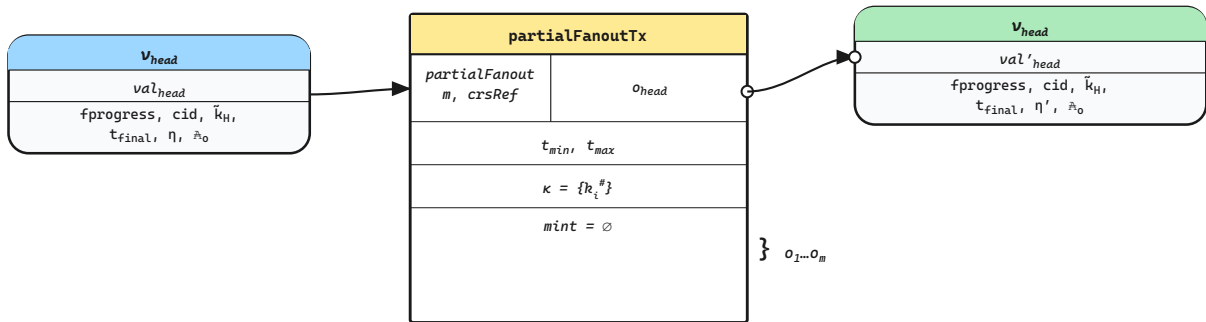


Figure 11: *partialFanout* transaction spending the `fanoutProgress` head output, distributing outputs $o_1 \dots o_m$, and producing a new `fanoutProgress` head output with updated accumulator η' .

5.8.2 Final Partial Fan-Out Transaction

Once all UTXOs except the last batch have been distributed via `partialFanout` steps, the final step burns all head tokens and distributes the remaining UTXOs. A final partial fanout transaction (see Figure 12) consists of:

- one input spending from ν_{head} in state `fanoutProgress`, and
- outputs $o_1 \dots o_m$ distributing the remaining UTXOs (no continuing head output).

The state-machine validator ν_{head} is spent with $\rho_{\text{head}} = (\text{finalPartialFanout}, m, \pi, \text{crsRef})$, where m is the number of UTXO outputs, π is the KZG membership witness, and `crsRef` is the CRS reference. The validator checks:

1. $m > 0$ (prevents fund theft via a zero-output proof bypass).
2. State is advanced from `fanoutProgress` to terminal state `final`:

$$(\text{fanoutProgress}, \text{cid}, \underline{k}_{\text{H}}, t_{\text{final}}, \eta, \mathbb{A}_{\text{O}}) \xrightarrow[m, \pi]{\text{finalPartialFanout}} \text{final}$$

3. All head tokens are burnt $|\{\text{cid} \mapsto \cdot \mapsto -1\} \in \text{mint}| = n + 1$.
4. Transaction is posted after contestation deadline $t_{\text{min}} > t_{\text{final}}$.
5. The m distributed outputs are verified as members of the accumulator η using the membership witness π :

$$\text{accVerify}(\eta, \{o_1, \dots, o_m\}, \pi) = \text{true}$$

6. Value is conserved:

$$\text{val}_{\text{head}}^{\text{in}} = \bigoplus_{i=1}^m \text{val}(o_i) \oplus \text{val}_{\text{burned}} \oplus \mathbb{A}_{\text{O}}$$

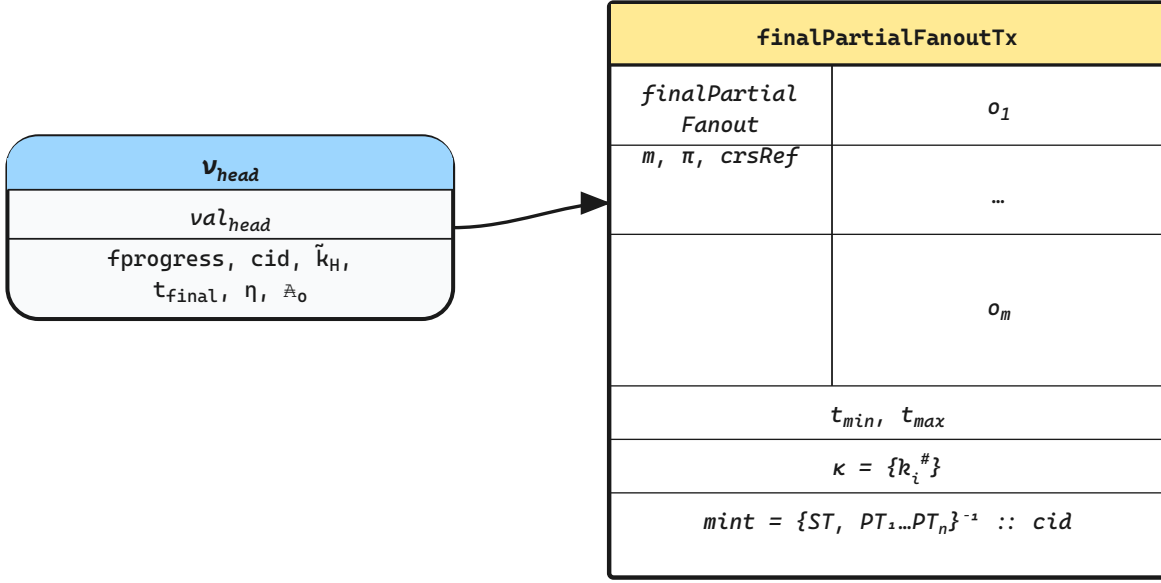


Figure 12: *finalPartialFanout* transaction spending the *fanoutProgress* head output, distributing the final batch of UTxOs $o_1 \dots o_m$, and burning all head tokens to reach *final*.

The $\mu_{head}(\phi_{seed})$ minting policy governs the burning of tokens via redeemer *burn* that:

1. All tokens in *mint* need to be of negative quantity $\forall \{cid \mapsto \cdot \mapsto q\} \in mint : q < 0$.

6 Off-Chain Protocol

This section describes the actual Coordinated Hydra Head protocol, an even more simplified version of the original publication [8]. See the protocol overview in Section 2 for an introduction and notable changes to the original protocol. While the on-chain part already describes the full life-cycle of a Hydra head on-chain, this section completes the picture by defining how the protocol behaves off-chain and notably the relationship between on- and off-chain semantics. Participants of the protocol are also called Hydra head members, parties or simply protocol actors. The protocol is specified as a reactive system that processes three kinds of inputs:

1. On-chain protocol transactions as introduced in Section 5, which are posted to the mainchain and can be observed by all actors:
 - **initialTx**: opens a head
 - **depositTx**: some UTxO was deposited to be incremented
 - **recoverTx**: deposited UTxO was recovered
 - **incrementTx**: adds UTxO to an open head
 - **decrementTx**: removes UTxO from an open head
 - **closeTx**: closes a head

- **contestTx**: contests a closed head
- **fanoutTx**: distributes all UTxOs from a closed head in one step
- **partialFanoutTx**: distributes a subset of UTxOs, transitioning to **fanoutProgress** (intermediate)
- **finalPartialFanoutTx**: distributes the last subset and burns tokens (terminal from **fanoutProgress**)

Also, a special input when time advanced on chain may be used:

- **tick**: time advanced on chain
2. Off-chain network messages sent between protocol actors (parties):
 - **reqTx**: to request a transaction to be included in the next snapshot
 - **reqDec**: to request exclusion of UTxO via a decommit transaction
 - **reqSn**: to request a snapshot to be created & signed by every head member
 - **ackSn**: to acknowledge a snapshot by replying with their signatures
 3. Commands issued by the participants themselves or on behalf of end-users and clients
 - **init**: to open a head
 - **close**: to request closure of an open head

The behavior is fully specified in Figure 13, while the following paragraphs introduce notation, explain variables and walk-through the protocol flow.

6.1 Assumptions

On top of the statements of the protocol setup in Section 4, the off-chain protocol logic relies on these assumptions:

- Every network message received from a specific party is checked for authentication. An implementation of the specification needs to find a suitable means of authentication, either on the communication channel or for individual messages. Unauthenticated messages must be dropped.
- The head protocol gets correctly (and with completeness) notified about observed transactions on-chain belonging to the respective head instance.
- All inputs are processed to completion, i.e. run-to-completion semantics and no preemption.
- Inputs are deduplicated. That is, any two identical inputs must not lead to multiple invocations of the handling semantics.
- Given the specification, inputs may pile up forever and implementations need to consider these situations (i.e. potential for DoS). A valid reaction to this would be to just drop these inputs. Note that, from a security standpoint, these situations are identical to a non-collaborative peer and closing the head is also a possible reaction.

move/merge
with pro-
tocol
setup?

- The lifecycle of a Hydra head on-chain does not cross (hard fork) protocol update boundaries. Note that these inputs are announced in advance hence it should be possible for implementations to react in such a way as to expedite closing of the head before such a protocol update. This further assumes that the contestation period parameter is picked accordingly.

6.2 Notation

missing:
apply tx

- **on event** specifies how the protocol reacts on a given input *event*. Further information may be available from the constituents of *event* and origin of the input.
- **require** p means that boolean expression $p \in \mathbb{B}$ must be satisfied for the further execution of a routine, while discontinued on $\neg p$. A conservative protocol actor could interpret this as a reason to close the head.
- **wait** p is a non-blocking wait for boolean predicate $p \in \mathbb{B}$ to be satisfied. On $\neg p$, the execution of the routine is stopped, queued, and reactivated at latest when p is satisfied.
- **multicast** msg means that a message msg is (channel-) authenticated and sent to all participants of this head, including the sender.
- **postTx** tx has a party create transaction tx , potentially from some data, and submit it on-chain. See Section 5 for individual transaction details.
- **output event** signals an observation of *event*, which is used in the security definition and proofs of Section 7. This keyword can be ignored when implementing the protocol.

6.3 Variables

Besides parameters agreed in the protocol setup (see Section 4), a party's local state consists of the following variables:

- $\hat{v}$: Last seen open state version.
- $\hat{s}$: Sequence number of latest seen snapshot.
- $\hat{\Sigma} \in (\mathbb{N} \times \mathbb{H})^*$: Accumulator of signatures of the latest seen snapshot, indexed by parties.
- $\hat{\mathcal{L}}$: UTxO set representing the local ledger state resulting from applying $\hat{\mathcal{T}}$ to $\bar{S}.U$ to validate requested transactions.
- $\hat{\mathcal{T}} \in \mathcal{T}^*$: List of transactions applied locally and pending inclusion in a snapshot (if this party is the next leader).
- $tx_\alpha \in \mathcal{T}$: Pending deposit transaction¹ to be used for incrementing the head state.
- $tx_\omega \in \mathcal{T}$: Pending decrement transaction, whose outputs are to be withdrawn from the head.
- $\mathcal{D}$: Set of deposit objects tracking deposit transactions with their status.

¹In fact this would only need to be a transaction id to look up the corresponding deposit in $\mathcal{D}$

- $\bar{\mathcal{S}}$: Snapshot object of the latest confirmed snapshot which contains:

$\bar{\mathcal{S}}.v$	snapshot version
$\bar{\mathcal{S}}.s$	snapshot number
$\bar{\mathcal{S}}.\mathcal{T}$	list of transactions relating this snapshot to the previous
$\bar{\mathcal{S}}.U$	snapshotted UTxO set
$\bar{\mathcal{S}}.U_\alpha$	pending UTxO to increment
$\bar{\mathcal{S}}.\phi_\alpha$	deposit the pending UTxO to increment comes from
$\bar{\mathcal{S}}.U_\omega$	pending UTxO to decrement
$\bar{\mathcal{S}}.\eta'^\#$	hash of the accumulator commitment over the snapshotted UTxO set

where constructor $\text{snObj}(v, n, T, U, U_\alpha, U_\omega)$ initializes a new snapshot object with $\bar{\mathcal{S}}.\eta'^\# = \perp$.

Additionally, deposit objects are created using $\text{depositObj}(U, t_{\text{created}}, t_{\text{deadline}}, \text{status})$ where status can be Inactive, Active, or Expired.

6.4 Protocol flow

6.4.1 Initializing the head

init. Before a head can be initialized, all parties need to exchange and agree on protocol parameters during the protocol setup phase (see Section 4), so we can assume the public Cardano keys k_C^{setup} , Hydra keys $\tilde{k}_H^{\text{setup}}$, as well as the contestation period $T_{\text{contest}}^{\text{setup}}$ are available. One of the clients then can start head initialization using the **init** command, which will result in an *init* transaction being posted. Not strictly a protocol parameter, but the deposit period T_{deposit} would also be made available from configuration.

initialTx. All parties will receive this *init* transaction and validate announced parameters against the pre-agreed *setup* parameters, as well as the structure of the transaction and the minting policy used. This is a vital step to ensure the initialized Head is valid, which cannot be checked completely on-chain (see also Section 5.1). Importantly, parties must verify that the transaction mints the correct tokens (one state thread token and one participation token per head member). This also helps in to distinguish *init* transactions from *increment* or *decrement* transactions, which may produce similar outputs without minting.

Since the *init* transaction directly opens the head with an empty UTxO set, the parties initialize their local state upon observing it: the local ledger state $\hat{\mathcal{L}} = \emptyset$, the seen transaction set $\hat{\mathcal{T}} = \emptyset$, the seen head state version $\hat{v} = 0$, and the snapshot number $\hat{s} = 0$. No deposit transaction $\text{tx}_\alpha = \perp$ and no decrement transaction $\text{tx}_\omega = \perp$ are pending, and the last confirmed snapshot is initialized accordingly $\bar{\mathcal{S}} \leftarrow \text{snObj}(0, 0, [], \emptyset, \emptyset, \emptyset)$.

6.4.2 Processing transactions off-chain

Transactions are announced and captured in so-called snapshots. Parties generate snapshots in a strictly sequential round-robin manner. The party responsible for issuing the i^{th} snapshot is the *leader* of the i^{th} snapshot. Leader selection is round-robin per the k_H from the protocol setup.

While the frequency of snapshots in the general Head protocol [8] was configurable, the Coordinated Head protocol does specify a snapshot to be created after each transaction.

reqTx. Upon receiving request $(\mathbf{reqTx}, \text{tx})$, the transaction is applied to the *local* ledger state $\hat{\mathcal{L}} \circ \text{tx}$. If not applicable yet, the protocol does **wait** to retry later or eventually marks this transaction as invalid (see assumption about events piling up). After applying and if there is no current snapshot in flight ($\hat{s} = \bar{\mathcal{S}}.s$) and the receiving party $\mathbf{p}_i$ is the next snapshot leader, a message to request snapshot signatures **reqSn** is sent.

reqDec. Upon receiving request $(\mathbf{reqDec}, \text{tx}_\omega)$, the transaction is checked against the *local* ledger state and if it is not applicable yet or another deposit or decommit is pending still, the protocol does **wait** to retry later or eventually marks the decommit as invalid. After applying tx , its outputs are removed from *local* ledger state $\hat{\mathcal{L}}$ so that they are not available any more and the decommit transaction is kept in the local state (tx_ω). If there is no current snapshot in flight ($\hat{s} = \bar{\mathcal{S}}.s$) and the receiving party $\mathbf{p}_i$ is the next snapshot leader, a message to request snapshot signatures **reqSn** containing the decrement transaction tx_ω is sent.

depositTx. Upon observing a deposit transaction, each party records the deposit index by deposit transaction id to their local deposit registry $\mathcal{D}$ with status Inactive. The deposit contains deposited UTxO U , creation time t_{created} , and deadline t_{deadline} .

recoverTx. Upon observing a recover transaction, each party drops the corresponding entry from its deposit registry $\mathcal{D}$.

tick. Whenever time advances (on-chain) to point t , parties update the status of deposits in $\mathcal{D}$ accordingly using the configured deposit period T_{deposit} :

- Expired when deadline passed (or too soon): $t > t_{\text{deadline}} - T_{\text{deposit}}$
- Active when deposit settled enough: $t > t_{\text{created}} + T_{\text{deposit}}$

When deposits become **Active** and no other deposit / decommit is pending, and the party is the next snapshot leader, it may request a new snapshot including the deposit transaction tx_α .

reqSn. Upon receiving request $(\mathbf{reqSn}, v, s, \underline{\text{tx}}_{\text{req}}, \text{tx}_\alpha, \text{tx}_\omega)^2$ from party $\mathbf{p}_j$, the receiving $\mathbf{p}_i$ **requires** that only a deposit or decommit may be pending, and that v refers to the current open state version, s is the next snapshot number and that party $\mathbf{p}_j$ is responsible for leading its creation. Party $\mathbf{p}_i$ may have to wait until the previous snapshot is confirmed ($\bar{\mathcal{S}}.s = \hat{s}$). Furthermore, the protocol validates the snapshot request by:

1. If a decommit is requested: verify the transaction is applicable to the last confirmed UTxO set and update the active utxo set with it

²Snapshot requests with only transaction identifiers and output references are possible if all parties keep an index of previously seen transactions and their identifiers.

2. If a decommit is requested: verify it produces at least one output, since *decrement* materializes the decommit outputs and requires one
3. If a deposit is requested: verify the corresponding deposit is Active and update the active utxo set with it
4. If we are on the same version as the last snapshot, any requested decommit or deposit must match the last snapshot.
5. Verify all requested transactions tx_{req} are applicable to the active UTxO set

Only then, $\mathbf{p}_i$ increments their seen-snapshot counter $\hat{s}$, resets the signature accumulator $\hat{\Sigma}$, and computes the UTxO set of the new local snapshot as $U \leftarrow U_{\text{active}} \circ \text{tx}_{\text{req}}$. Then, $\mathbf{p}_i$ creates a signature σ_i using their signing key $k_{\text{H}}^{\text{sig}}$ on a message comprised by the cid, the new snapshot number $\hat{s}$, the new η resulting from canonically combining U (see Section 5.6 for details), and either η_α or η_ω derived from deposited U_α or decommit transaction tx_ω respectively. Note that η_α covers both U_α and the deposit ϕ_α it comes from (see Section 5.4), so the resulting signature authorizes that one deposit and no other recording the same UTxO. The signature is sent to all head members via message $(\text{ackSn}, \hat{s}, \sigma_i)$. Finally, the local ledger state $\mathcal{L}$ and pending transaction set $\hat{\mathcal{T}}$ get pruned by re-applying all locally pending transactions $\hat{\mathcal{T}}$ to the just requested snapshot's UTxO set iteratively and ultimately yielding a “pruned” version of $\hat{\mathcal{T}}$ and $\mathcal{L}$.

ackSn. Upon receiving acknowledgment $(\text{ackSn}, s, \sigma_j)$, all participants **require** that it is from an expected snapshot (either the last seen $\hat{s}$ or $+1$), potentially **wait** for the corresponding **reqSn** such that $\hat{s} = s$ and **require** that the signature is not yet included in $\hat{\Sigma}$. They store the received signature in the signature accumulator $\hat{\Sigma}$, and if the signature from each party has been collected, $\mathbf{p}_i$ aggregates the multisignature $\tilde{\sigma}$ and **require** it to be valid (constructing the signed message as in **reqSn**). If everything is fine, the snapshot can be considered confirmed by creating the snapshot object $\mathcal{S} \leftarrow \text{snObj}(\hat{v}, \hat{s}, \hat{\mathcal{T}}, \hat{U}, U_\alpha, \text{outputs}(\text{tx}_\omega))$ and storing the multi-signature $\tilde{\sigma}$ in it for later reference. In case there is a pending decommit, any participant can now submit a *decrement* transaction by providing the just confirmed snapshot with its accumulator commitment η' for the updated UTxO set. If, however, there was a pending deposit, any participant can now submit an **incrementTx** by providing the confirmed snapshot with its accumulator commitment η' for the updated UTxO set. Lastly, if $\mathbf{p}_i$ is the next snapshot leader and there are already transactions to snapshot in $\hat{\mathcal{T}}$, a corresponding **reqSn** is distributed.

decrementTx. Upon observing the *decrement* transaction, which removed outputs U from the head, the corresponding pending decommit transaction is cleared and the observed version v is used for future snapshots by setting $\hat{v} \leftarrow v$. Note that the version of the open head state is incremented on each *decrement* transaction as described in Section 5.5.

incrementTx. Upon observing the *increment* transaction, which added outputs U to the head, the local ledger state $\mathcal{L}$ is extended with the newly added UTxO while the pending increment state U_α is cleared. Also the observed version v is used for future snapshots by setting $\hat{v} = v$. Note that the version of the open head state is incremented on each *increment* transaction as described in Section 5.4

6.4.3 Closing the head

close. In order to close a head, a client issues the `close` input which uses the latest confirmed snapshot $\bar{s}$ to construct the accumulator commitment η' for the closing UTxO state and the certificate ξ using the corresponding multi-signature. With these, the *close* transaction can be constructed and posted. See Section 5.6 for details about this transaction.

closeTx/contestTx. When a party observes the head getting closed or contested, the η -state extracted from the *close* or *contest* transaction represents the latest head status that has been aggregated on-chain so far (by a sequence of *close* and *contest* transactions). If the last confirmed (off-chain) snapshot is newer than the observed (on-chain) snapshot number s_c , an updated accumulator commitment η' and certificate ξ are constructed and posted in a *contest* transaction (see Section 5.7).

fanoutTx. Upon observing a *fanout* transaction, all UTxOs are distributed and the head transitions to final state.

partialFanoutTx. When a head needs to be finalized, the node first attempts to post a single *fanout* transaction distributing all UTxOs at once. If the full UTxO set does not fit within the transaction size or script execution budget, the node falls back to partial fanout: it performs a binary search over batch sizes to find the largest batch that fits within protocol limits, minimising the total number of fanout steps. Concretely, for a remaining set of N UTxOs, it searches $[1, N - 1]$ using ceiling-division midpoints (biasing toward larger batches) and selects the largest n for which the transaction satisfies both the size limit and the script execution budget.

A batch of size $n < N$ is posted as an intermediate *partialFanout* transaction, which distributes n UTxOs, updates the accumulator commitment to cover only the remaining $N - n$ UTxOs, and transitions the head to the `fanoutProgress` state. The procedure repeats: after each *partialFanout* observation the node again attempts to finalize all remaining UTxOs in one step; if they still do not fit, another binary search determines the next batch size. *finalPartialFanout* is used for the last batch when all remaining UTxOs fit in a single transaction, burning all head tokens and transitioning to `final`.

6.5 Rollbacks and protocol changes

The overall life-cycle of the Head protocol is driven by on-chain inputs (see introduction of Section 6) which stem from observing transactions on the mainchain. Most blockchains, however, do only provide *eventual* consistency. The consensus algorithm ensures a consistent view of the history of blocks and transactions between all parties, but this so-called *finality* is only achieved after some time and the local view of the blockchain history may change until that point.

On Cardano with its Ouroboros consensus algorithm, this means that any local view of the mainchain may not be the longest chain and a node may switch to a longer chain, onto another fork. This other version of the history may not include what was previously observed and hence, any tracking state needs to be updated to this “new reality”. Practically, this means that an observer of the blockchain sees a *rollback* followed by rollforwards.

For the Head protocol, this means that chain events like `closeTx` may be observed a second time. Hence, it is crucial, that the local state of the Hydra protocol is kept in sync and also rolled back accordingly to be able to observe and react to these events the right way, e.g. correctly contesting this `closeTx` if need be.

Explain why rollbacks are no problem to increment/decrement

Write about contestation deadline vs. rollbacks

The rollback handling can be specified fully orthogonal on top of the nominal protocol behavior, if the chain provides strictly monotonically increasing points p on each chain event via a new or wrapped **rollforward** event and **rollback** event with the point to which a rollback happened:

rollforward. On every chain event that is paired or wrapped in a rollforward event (**rollback**, p) with point p , protocol participants store their head state indexed by this point in a history Ω of states $\Delta \leftarrow (\hat{v}, \hat{s}, \hat{u}, \hat{\Sigma}, \hat{\mathcal{L}}, \hat{\mathcal{T}}, \hat{\mathcal{S}})$ and $\Omega' = (p, \Delta) \cup \Omega$.

rollback. On a rollback (**rollback**, p_{rb}) to point p_{rb} , the corresponding head state Δ need to be retrieved from Ω , with the maximal point $p \leq p_{rb}$, and all entries in Ω with $p > p_{rb}$ get removed.

This will essentially reset the local head state to the right point and allow the protocol to progress through the life-cycle normally. Most stages of the life-cycle are unproblematic if they are rolled back, as long as the protocol logic behaves as in the nominal case.

Since the head is directly opened by the *init* transaction, every rollback of on-chain state is effectively a rollback “past open”. However, because the head opens with an empty UTxO set and funds are only added via deposits, the critical concern becomes rollbacks of deposit transactions. The deposit settlement period T_{deposit} (see Section 6) ensures that a deposit is only considered **Active** — and thus eligible for incrementing into the head — after sufficient time has elapsed since its creation on-chain. This means that by the time a deposit is incremented, its on-chain transaction is sufficiently settled and unlikely to be rolled back, preventing inconsistency between the on-chain and off-chain state.

Coordinated Hydra Head

```

on (init) from client
   $n \leftarrow |k_H^{setup}|$ 
   $\tilde{k}_H \leftarrow \text{MS-AVK}(k_H^{setup})$ 
   $\tilde{k}_C \leftarrow k_C^{setup}$ 
   $T_{\text{contest}} \leftarrow T_{\text{contest}}^{setup}$ 
   $T_{\text{deposit}} \leftarrow T_{\text{deposit}}^{setup}$ 
  postTx (init,  $n, \tilde{k}_H, \tilde{k}_C, T_{\text{contest}}$ )

on (initialTx, cid,  $\phi_{\text{seed}}, n, \tilde{k}_H, \tilde{k}_C^{\#}, T_{\text{contest}}$ ) from chain
  require  $\tilde{k}_H = \text{MS-AVK}(k_H^{setup})$ 
  require  $\tilde{k}_C^{\#} = [\text{hash}(k) \mid \forall k \in k_C^{setup}]$ 
  require  $T_{\text{contest}} = T_{\text{contest}}^{setup}$ 
  require cid = hash( $\mu_{\text{head}}(\phi_{\text{seed}})$ )
   $\hat{\mathcal{L}} \leftarrow \emptyset$ 
   $\tilde{s} \leftarrow \text{snObj}(0, 0, [], \emptyset, \emptyset, \emptyset)$ 
   $\hat{v}, \hat{s} \leftarrow 0$ 
   $\hat{\mathcal{T}} \leftarrow \emptyset$ 
   $\text{tx}_{\omega} \leftarrow \perp$ 
   $\text{tx}_{\alpha} \leftarrow \perp$ 

```

```

on (reqTx, tx) from pj
  wait  $\hat{\mathcal{L}} \circ \text{tx} \neq \perp$ 
   $\hat{\mathcal{L}} \leftarrow \hat{\mathcal{L}} \circ \text{tx}$ 
   $\hat{\mathcal{T}} \leftarrow \hat{\mathcal{T}} \cup \{\text{tx}\}$ 
  if  $\hat{s} = \tilde{s}.s \wedge \text{leader}(\tilde{s}.s + 1) = i$ 
    if  $\text{tx}_{\alpha} = \perp \wedge \text{tx}_{\omega} = \perp \wedge \tilde{s}.U_{\alpha} = \emptyset$ 
       $\text{tx}_{\alpha} \leftarrow \text{oldest } D \in \mathcal{D} \text{ with } D.\text{status} = \text{Active}$ 
      multicast (reqSn,  $\hat{v}, \tilde{s}.s + 1, \hat{\mathcal{T}}, \text{tx}_{\alpha}, \text{tx}_{\omega}$ )

on (reqDec, tx) from pj
  wait  $U_{\alpha} = \emptyset \wedge \text{tx}_{\omega} = \perp \wedge \hat{\mathcal{L}} \circ \text{tx} \neq \perp$ 
   $\hat{\mathcal{L}} \leftarrow \hat{\mathcal{L}} \circ \text{tx} \setminus \text{outputs}(\text{tx})$ 
   $\text{tx}_{\omega} \leftarrow \text{tx}$ 
  if  $\hat{s} = \tilde{s}.s \wedge \text{leader}(\tilde{s}.s + 1) = i$ 
    multicast (reqSn,  $\hat{v}, \tilde{s}.s + 1, \hat{\mathcal{T}}, \perp, \text{tx}_{\omega}$ )

on (reqSn,  $v, s, \text{tx}_{\text{req}}, \text{tx}_{\alpha}, \text{tx}_{\omega}$ ) from pj
  require  $v = \hat{v} \wedge s = \hat{s} + 1 \wedge \text{leader}(s) = j$ 
  wait  $\hat{s} = \tilde{s}.s \wedge v = \hat{v}$ 
  require  $\text{tx}_{\omega} = \perp \vee \text{tx}_{\alpha} = \perp$ 
  if  $\text{tx}_{\omega} \neq \perp$ 
    if  $v = \tilde{s}.v \wedge \tilde{s}.U_{\omega} \neq \perp$ 
      require  $\tilde{s}.U_{\omega} = \text{outputs}(\text{tx}_{\omega})$ 
    else
      require  $\tilde{s}.U \circ \text{tx}_{\omega} \neq \perp$ 
       $U_{\text{active}} \leftarrow \tilde{s}.U \circ \text{tx}_{\omega} \setminus \text{outputs}(\text{tx}_{\omega})$ 
  if  $\text{tx}_{\alpha} \neq \perp$ 
     $\mathcal{D} \leftarrow \mathcal{D}[\text{tx}_{\alpha}]$ 
    require  $\mathcal{D}.\text{status} \neq \text{Expired}$ 
    wait  $\mathcal{D}.\text{status} = \text{Active}$ 
    if  $v = \tilde{s}.v \wedge \tilde{s}.U_{\alpha} \neq \perp$ 
      require  $\tilde{s}.U_{\alpha} = \mathcal{D}.U$ 
    else
       $U_{\alpha} \leftarrow \mathcal{D}.U$ 
       $U_{\text{active}} \leftarrow U_{\text{active}} \cup U_{\alpha}$ 
  require  $U_{\text{active}} \circ \text{tx}_{\text{req}} \neq \perp$ 
   $U \leftarrow U_{\text{active}} \circ \text{tx}_{\text{req}}$ 
   $\hat{s} \leftarrow s$ 
   $\eta' \leftarrow \text{accUTxO}(U)$ 
   $\eta'^{\#} \leftarrow (\eta')^{\#}$ 
   $\sigma_i \leftarrow \text{MS-Sign}(k_H^{sig}, (\text{cid} || v || \hat{s} || \eta'^{\#}))$ 
   $\hat{\Sigma} \leftarrow \emptyset$ 
  multicast (ackSn,  $\hat{s}, \sigma_i$ )
   $\forall \text{tx} \in \text{tx}_{\text{req}} : \text{output}(\text{seen}, \text{tx})$ 
   $\hat{\mathcal{L}} \leftarrow U$ 
   $X \leftarrow \hat{\mathcal{T}}$ 
   $\hat{\mathcal{T}} \leftarrow \emptyset$ 
  for  $\text{tx} \in X : \hat{\mathcal{L}} \circ \text{tx} \neq \perp$ 
     $\hat{\mathcal{T}} \leftarrow \hat{\mathcal{T}} \cup \{\text{tx}\}$ 
     $\hat{\mathcal{L}} \leftarrow \hat{\mathcal{L}} \circ \text{tx}$ 

on (ackSn,  $s, \sigma_j$ ) from pj
  require  $s \in \{\hat{s}, \hat{s} + 1\}$ 
  wait  $\hat{s} = s$ 
  require  $(j, \cdot) \notin \hat{\Sigma}$ 
   $\hat{\Sigma}[j] \leftarrow \sigma_j$ 
  if  $\forall k \in [1..n] : (k, \cdot) \in \hat{\Sigma}$ 
     $\tilde{\sigma} \leftarrow \text{MS-ASig}(k_H^{setup}, \hat{\Sigma})$ 
     $\eta' \leftarrow \text{accUTxO}(U)$ 
     $\eta'^{\#} \leftarrow (\eta')^{\#}$ 
    require  $\text{MS-Verify}(\tilde{k}_H, (\text{cid} || \hat{v} || \hat{s} || \eta'^{\#}), \tilde{\sigma})$ 
     $\tilde{s} \leftarrow \text{snObj}(\hat{v}, \hat{s}, \hat{\mathcal{T}}, \hat{U}, U_{\alpha}, U_{\omega})$ 
     $\tilde{s}.\sigma \leftarrow \tilde{\sigma}$ 
     $\forall \text{tx} \in \mathcal{T}_{\text{req}} : \text{output}(\text{conf}, \text{tx})$ 
    if  $\tilde{s}.U_{\omega} \neq \perp$ 
      postTx (decrementTx,  $\hat{v}, \hat{s}, \eta'^{\#}, \tilde{s}.\sigma$ )
    if  $\tilde{s}.U_{\alpha} \neq \perp$ 
      postTx (incrementTx,  $\hat{v}, \hat{s}, \eta'^{\#}, \tilde{s}.\sigma$ )
    if  $\text{leader}(s + 1) = i \wedge \hat{\mathcal{T}} \neq \emptyset$ 
      if  $\text{tx}_{\alpha} = \perp \wedge \text{tx}_{\omega} = \perp \wedge \tilde{s}.U_{\alpha} = \emptyset$ 
         $\text{tx}_{\alpha} \leftarrow \text{oldest } D \in \mathcal{D} \text{ with } D.\text{status} = \text{Active}$ 
      multicast (reqSn,  $\hat{v}, \tilde{s}.s + 1, \hat{\mathcal{T}}, \text{tx}_{\alpha}, \text{tx}_{\omega}$ )

on (depositTx,  $\text{tx}_{\alpha}, U, t_{\text{created}}, t_{\text{deadline}}$ ) from chain
   $\mathcal{D} \leftarrow \mathcal{D} \cup (\text{tx}_{\alpha}, \text{depositObj}(U, t_{\text{created}}, t_{\text{deadline}}, \text{Inactive}))$ 

on (recoverTx,  $\text{tx}_{\alpha}$ ) from chain
   $\mathcal{D} \leftarrow \mathcal{D} \setminus (\text{tx}_{\alpha}, \cdot)$ 

on (decrementTx,  $U, v$ ) from chain
   $\hat{s} \leftarrow \tilde{s}.s$ 
   $\hat{v} \leftarrow v$ 
   $\text{tx}_{\omega} \leftarrow \perp$ 
  if  $\text{leader}(\tilde{s}.s + 1) = i \wedge \hat{\mathcal{T}} \neq \emptyset$ 
    multicast (reqSn,  $\hat{v}, \tilde{s}.s + 1, \hat{\mathcal{T}}, \text{tx}_{\alpha}, \perp$ )

on (incrementTx,  $U, v$ ) from chain
   $\hat{s} \leftarrow \tilde{s}.s$ 
   $\hat{v} \leftarrow v$ 
   $\text{tx}_{\alpha} \leftarrow \perp$ 
   $\hat{\mathcal{L}} \leftarrow \hat{\mathcal{L}} \cup U$ 
  if  $\text{leader}(\tilde{s}.s + 1) = i \wedge \hat{\mathcal{T}} \neq \emptyset$ 
    multicast (reqSn,  $\hat{v}, \tilde{s}.s + 1, \hat{\mathcal{T}}, \perp, \perp$ )

on (tick,  $t$ ) from chain
  for  $D \in \mathcal{D}$ 
    if  $t > D.\text{deadline} - T_{\text{deposit}}$ 
       $D.\text{status} \leftarrow \text{Expired}$ 
    else if  $t > D.\text{created} + T_{\text{deposit}}$ 
       $D.\text{status} \leftarrow \text{Active}$  if  $\text{tx}_{\alpha} = \perp \wedge \text{tx}_{\omega} = \perp$ 
         $\text{tx}_{\alpha} \leftarrow D$ 
  if  $\exists D \in \mathcal{D} : D.\text{status} = \text{Active}$ 
    if  $\text{tx}_{\alpha} \neq \perp \wedge \text{tx}_{\omega} = \perp \wedge \hat{s} = \tilde{s}.s \wedge \text{leader}(\tilde{s}.s + 1) = i$ 
      multicast (reqSn,  $\hat{v}, \tilde{s}.s + 1, \hat{\mathcal{T}}, \text{tx}_{\alpha}, \perp$ )

```

<pre> on (close) from client $\eta'^{\#} \leftarrow \bar{S}.\eta^{\#}$ $\xi \leftarrow \bar{S}.\sigma$ postTx (close, $\hat{v}$, $\bar{S}.v$, $\bar{S}.s$, $\eta'^{\#}$, ξ) </pre>	<pre> on (closeTx, $\eta'^{\#}$) $\vee$ (contestTx, s_c, $\eta'^{\#}$) from chain if $\bar{S}.s > s_c$ $\eta'^{\#} \leftarrow \bar{S}.\eta^{\#}$ $\xi \leftarrow \bar{S}.\sigma$ postTx (contest, $\hat{v}$, $\bar{S}.v$, $\bar{S}.s$, $\eta'^{\#}$, ξ) </pre>
--	--

Figure 13: Head-protocol machine for the *coordinated head* from the perspective of party p_i .

7 Security (WIP — Iteration 1)

Adversaries:

Active Adversary. An *active adversary* $\mathcal{A}$ has full control over the protocol, i.e., he is fully unrestricted in the above security game.

Network Adversary. A *network adversary* $\mathcal{A}_0$ does not corrupt any head parties, eventually delivers all sent network messages (i.e., does not drop any messages), and does not cause the **close** event. Apart from this restriction, the adversary can act arbitrarily in the above experiment.

Random variables:

- $\hat{S}_i$: the set of transactions tx for which party p_i , *while uncorrupted*, output (**seen**, tx);
- $\bar{C}_i$: the set of transactions tx for which party p_i , *while uncorrupted*, output (**conf**, tx);
- $\bar{\Sigma}_i$: latest snapshot (s, U) that party p_i performed *while uncorrupted*: output (**snap**, (s, U));
- H_{cont} : the set of (at the time) uncorrupted parties who produced ξ upon close/contest request and ξ was applied to correct η ; and
- $\mathcal{H}$: the set of parties that remain uncorrupted.

Security conditions / events:

- **CONSISTENCY (HEAD)**: In presence of an active adversary, the following condition holds at any point in time: For all i, j , $U_0 \circ (\bar{C}_i \cup \bar{C}_j) \neq \perp$, i.e., no two uncorrupted parties see conflicting transactions confirmed.
- **OBLIVIOUS LIVENESS (HEAD)**: Consider any protocol execution in presence of a network adversary wherein the head does not get closed for a sufficiently long period of time, and consider an honest party p_i who enters transaction tx by executing (**newTx**, tx) *each time after having finished a snapshot*.

Then the following eventually holds: $\text{tx} \in \bigcap_{i \in [n]} \bar{C}_i \vee \forall i : U_0 \circ (\bar{C}_i \cup \{\text{tx}\}) = \perp$, i.e., every party will observe the transaction confirmed or every party will observe the transaction in conflict with their confirmed transactions.³

³In particular, *liveness* expresses that the protocol makes progress under reasonable network conditions if no head parties get corrupted.

The security analysis is still **sketchy**, with the goal to make it more formal in upcoming iterations

Add security experiment

above this section there is no security game

- **SOUNDNESS (CHAIN):** In presence of an active adversary, the following condition is satisfied:
 $\exists \tilde{S} \subseteq \bigcap_{i \in \mathcal{H}} \hat{S}_i : U_{\text{final}} = U_0 \circ \tilde{S} \neq \perp$, i.e., the final UTxO set results from applying a set of transactions to U_0 that have been seen by all honest parties (whereas each such transaction applies conforming to the ledger rules).
- **COMPLETENESS (CHAIN):** In presence of an active adversary, the following condition holds:
 For $\tilde{S}$ as above, $\bigcup_{p_i \in H_{\text{cont}}} \bar{C}_i \subseteq \tilde{S}$, i.e., all transactions seen as confirmed by an honest party at the end of the protocol are considered.

Note that the original version of the coordinated head satisfies a stronger version of liveness which is important for the 'user experience' in the protocol:

- **LIVENESS (HEAD):** Consider any protocol execution in presence of a network adversary wherein the head does not get closed for a sufficiently long period of time, and consider an honest party p_i who enters transaction tx by executing (`newTx`, tx).
 Then the following eventually holds: $\text{tx} \in \bigcap_{i \in [n]} \bar{C}_i \vee \forall i : U_0 \circ (\bar{C}_i \cup \{\text{tx}\}) = \perp$, i.e., every party will observe the transaction confirmed or every party will observe the transaction in conflict with their confirmed transactions.⁴

7.1 Proofs

Consistency.

Lemma 1 (Consistency). *The coordinated head protocol satisfies the CONSISTENCY property.*

Proof. Observe that $\bar{C}_i \cup \bar{C}_j \subseteq \hat{S}_i$ since no transaction can be confirmed without every honest party signing off on it. Since parties do not sign conflicting transactions (see `reqSn`, 'wait'), we have $U_0 \circ \bar{C}_i \neq \perp$, $U_0 \circ \bar{C}_j \neq \perp$, and $U_0 \circ \hat{S}_i \neq \perp$. Thus, since $\bar{C}_i \cup \bar{C}_j \subseteq \hat{S}_i$ it follows that $U_0 \circ (\bar{C}_i \cup \bar{C}_j) \neq \perp$ $\square$

Oblivious Liveness. For all lemmas towards oblivious liveness, we assume the presence of a network adversary, and that the head does not get closed for a sufficiently long period of time. We call this the *liveness condition*.

Lemma 2. *Under the liveness condition, any snapshot issued as (`reqSn`, s, T) will eventually be confirmed in the sense that every party holds a valid multisignature on it.*

Proof. Consider a party p_i receiving message (`reqSn`, s, T). We demonstrate that p_i executes the code past the 'wait' instruction of the `reqSn` routine.

- **Passing the 'require' guard:** Note that the snapshot leader sends the request only if $\hat{s} = \bar{s}$, and for $s = \hat{s} + 1$. Thus, $\hat{s}_i = \hat{s}$ since p_i has already signed the snapshot for $\hat{s}$. The 'require' guard is thus satisfied for p_i .
- **Passing the 'wait' guard:** Since the snapshot leader sees $\hat{s} = \bar{s}$, also p_i will eventually see $\hat{s}_i = \bar{s}_i$. Furthermore, since all leaders are honest, it holds that $\hat{\mathcal{U}} \circ \mathcal{T}_{res} \neq \perp$ by construction.

⁴In particular, *liveness* expresses that the protocol makes progress under reasonable network conditions if no head parties get corrupted.

This implies that every party will eventually sign and acknowledge the newly created snapshot. Finally, the ‘require’ and ‘wait’ guards of the `ackSn` code will be passed by every party since an `ackSn` for snapshot number s can only be received for $s \in \{\hat{s}, \hat{s}+1\}$ as an acknowledgement can only be received for the current snapshot being worked on by p_i or a snapshot that is one step ahead—implying that everybody will hold a valid multisignature on the snapshot in consideration. $\square$

Lemma 3 (Eternal snapshot confirmation). *Under the liveness condition, as long as new transactions are issued, for any $k > 0$, every party eventually confirms a snapshot with sequence number $s = k$.*

Proof. By Lemma 2, any requested snapshot eventually gets confirmed, implying that the next leader observes $\hat{s} = \bar{s}$ and thus, in turn, issues a new snapshot. Thus, for any k , a snapshot is eventually confirmed. $\square$

Lemma 4 (Oblivious Liveness). *The coordinated head protocol satisfies the OBLIVIOUS LIVENESS property.*

Proof. Consider the first point in time where a transaction tx enters the system by some party p_i issuing `(newTx, tx)`, and consider the next point in time t when p_i issues a snapshot.

By Lemma 3, this snapshot will eventually be issued and confirmed by all parties.

Let $\hat{\mathcal{T}}$ be the transactions to be considered by p_i ’s snapshot: $\hat{\mathcal{L}} = \bar{U} \circ \hat{\mathcal{T}}$ where $\bar{U}$ is the snapshot prior to p_i ’s. Since p_i issues `(reqTx, tx)` after each snapshot, we have that, either,

- $\text{tx} \in \hat{\mathcal{T}}$, in which case $\text{tx} \in \bigcap_{i \in [n]} \bar{C}_i$ after everybody has completed this snapshot, or,
- $\text{tx} \notin \hat{\mathcal{T}}$, in which case $\hat{\mathcal{L}} \circ \text{tx} = \perp$ (tx is still in the wait queue of `(reqTx, tx)`). After everybody has completed this snapshot, it thus holds that $\forall i : U_0 \circ \bar{C}_i = \hat{\mathcal{L}}$, and thus, that $\forall i : U_0 \circ (\bar{C}_i \cup \{\text{tx}\}) = \perp$.

In both cases, the lemma follows. $\square$

Soundness and completeness.

Lemma 5 (Soundness). *The basic head protocol satisfies the SOUNDNESS property.*

Proof. Let T be the set of transactions such that $U_{\text{final}} = U_0 \circ T$. Since U_{final} is multi-signed, it holds that $T \subseteq \hat{S}_i$ (T is *seen*) by every honest party in the head. Furthermore, since honest signatures are only issued for valid transaction, $U_{\text{final}} \neq \perp$ (i.e., U_{final} is a valid state), and soundness follows. $\square$

Lemma 6 (Completeness). *The basic head protocol satisfies the COMPLETENESS property.*

Proof. Consider all parties $p_i \in H_{\text{cont}}$. Since the close/contest process finally accepts the latest multi-signed snapshot, it holds that $U_{\text{final}.s} \geq \max_{p_i \in H_{\text{cont}}} (\bar{s}_i)$, and thus that $\bigcup_{p_i \in H_{\text{cont}}} \bar{C}_i \subseteq \bigcap_{p_i \in \mathcal{H}} \hat{S}_i$, and completeness follows. $\square$

References

- [1] Extended UTXO-2 model. <https://github.com/hydra-supplementary-material/eutxo-spec/blob/master/extended-utxo-specification.pdf>.
- [2] A formal specification of the cardano ledger. <https://github.com/input-output-hk/cardano-ledger/releases/latest/download/shelley-ledger.pdf>.
- [3] A formal specification of the cardano ledger integrating plutus core. <https://github.com/input-output-hk/cardano-ledger/releases/latest/download/alonzo-ledger.pdf>.
- [4] Hydra repository. <https://github.com/input-output-hk/hydra>.
- [5] Nicola Atzei, Massimo Bartoletti, Stefano Lande, and Roberto Zunino. A formal model of Bitcoin transactions. In *Financial Cryptography and Data Security - 22nd International Conference, FC 2018, Nieuwpoort, Curaçao, February 26 - March 2, 2018, Revised Selected Papers*, pages 541–560, 2018.
- [6] Manuel M. T. Chakravarty, James Chapman, Kenneth MacKenzie, Orestis Melkonian, Michael Peyton Jones, and Philip Wadler. The extended UTxO model. In *4th Workshop on Trusted Smart Contracts*, 2020. http://fc20.ifca.ai/wtsc/WTSC2020/WTSC20_paper_25.pdf.
- [7] Manuel M. T. Chakravarty, James Chapman, Kenneth M. Mackenzie, Orestis Melkonian, Jann, Müller, Michael Peyton Jones, Polina Vinogradova, Philip Wadler, Joachim, and Zahnentferner. Utxoma: Utxo with multi-asset support. 2020.
- [8] Manuel MT Chakravarty, Sandro Coretti, Matthias Fitzi, Peter Gazi, Philipp Kant, Aggelos Kiayias, and Alexander Russell. Hydra: Fast isomorphic state channels. *Cryptology ePrint Archive*, 2020.
- [9] Kazuharu Itakura and Katsuhiko Nakamura. A public-key cryptosystem suitable for digital multisignatures. *NEC Research & Development*, (71):1–8, 1983.
- [10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology – ASIACRYPT 2010*, pages 177–194, 2010.
- [11] Silvio Micali, Kazuo Ohta, and Leonid Reyzin. Accountable-subgroup multisignatures: Extended abstract. pages 245–254, 2001.
- [12] Joachim Zahnentferner. An abstract model of UTxO-based cryptocurrencies with scripts. *IACR Cryptology ePrint Archive*, 2018:469, 2018.